The Cryptographic Atlas

A practical map of modern cryptographic primitives, protocols, guarantees, and design patterns.

Content was generated and revised with AI agents. Treat this PDF as educational draft material and verify technical claims against cited sources before relying on them.

Generated 2026-06-04. Source: https://christophelebrun.github.io/cryptographic-atlas/

Table of Contents

Introduction

Welcome to The Cryptographic Atlas

The Cryptographic Atlas is a living map of cryptographic concepts: what they do, what they do not do, what assumptions they rely on, and how they are commonly composed into larger systems.

This book is written for technical readers who need practical conceptual clarity before they evaluate designs, read protocol documentation, or talk with cryptographers.

Mission

The mission is to explain cryptographic building blocks as parts of a system design vocabulary. A reader should leave with sharper questions about threat models, assumptions, failure modes, maturity, and composition risks.

What this book is

This book is:

- an educational atlas of cryptographic ideas;
- a guide to categories, guarantees, and trade-offs;
- a place to compare primitives, protocols, proof systems, and design patterns;
- a reminder that real systems depend on more than mathematical primitives.

What this book is not

This book is not production cryptography guidance. It is not a source of copy-paste implementations, audited protocols, or deployment recipes.

Do not design or deploy custom cryptographic protocols without expert review.

The atlas is also not complete. See Editorial Maturity and Coverage for the current coverage assessment, known missing concepts, and source-depth priorities.

AI generation disclosure

The content in this book was generated and iteratively revised with AI agents. Treat it as educational draft material: verify technical claims against the cited sources, and do not rely on it as a substitute for expert cryptographic review.

How to read it

Start with the taxonomy, then move from goals to building blocks:

1. Identify the security goal.

2. Identify the assumptions and trust model.
3. Pick the primitive, proof system, protocol, or pattern being discussed.
4. Check what it does not provide.
5. Look for failure modes and composition risks.
6. Check its post-quantum posture and confidence model.

Core taxonomy

The atlas uses eight levels:

Level	What it describes	Examples
Security goals	Desired properties	confidentiality, integrity, anonymity
Assumptions and substrates	Mathematical or operational foundations	discrete logarithm, random oracle model, trusted setup
Basic primitives	Small cryptographic building blocks	hashes, commitments, signatures
Structured primitives	Building blocks with richer structure	homomorphic encryption, threshold signatures
Proof systems	Ways to prove statements under constraints	zero-knowledge proofs, range proofs
Protocols	Multi-step interactions between parties	MPC, secure aggregation, anonymous credentials
Systems	End-to-end applications	e-voting, private payments, anonymous airdrops
Design patterns	Reusable composition ideas	anonymous membership, delayed reveal, private aggregation

Concrete algorithms, named schemes, parameter families, and protocol suites are treated as instances of these concepts, not as a ninth top-level category. For example, Ed25519 is a concrete instance of digital signatures, Groth16 is a concrete instance of a proof system, and TLS 1.3 is a concrete protocol suite. See Concrete Algorithms and Schemes.

Motivating example: private voting

A private voting system may require anonymous eligibility, anti-double-vote protection, ballot secrecy, valid ballot proofs, private tallying, delayed reveal, and coercion mitigation. No single primitive provides all of these. The system must compose several tools, each with different assumptions and failure modes.

For example:

- anonymous credentials can help prove eligibility without directly revealing identity;
- nullifiers can help detect double use without naming the voter;
- commitments can bind a voter to a ballot before it is opened or tallied;
- zero-knowledge proofs can show that a hidden ballot is valid;
- homomorphic encryption or multi-party computation can support private tallying;
- coercion resistance requires protocol and user-experience properties beyond ballot secrecy.

The central lesson is composability: each tool contributes a narrow guarantee, and the system must make the gaps explicit.

Cross-cutting questions

For every major concept, ask:

- Is the construction quantum-vulnerable, plausibly post-quantum, or dependent on instantiation?
- Does confidence come from a mathematical assumption, public verification, one honest party, a `t-of-n` threshold, an honest majority, non-collusion, or a trusted issuer?
- Which part of the system has the weakest posture or most centralized confidence model?

Safety note

Cryptography is easy to misuse because guarantees are precise and conditional. This atlas tries to make those conditions visible.

Further reading

- Boneh and Shoup, "A Graduate Course in Applied Cryptography."
- Katz and Lindell, "Introduction to Modern Cryptography."

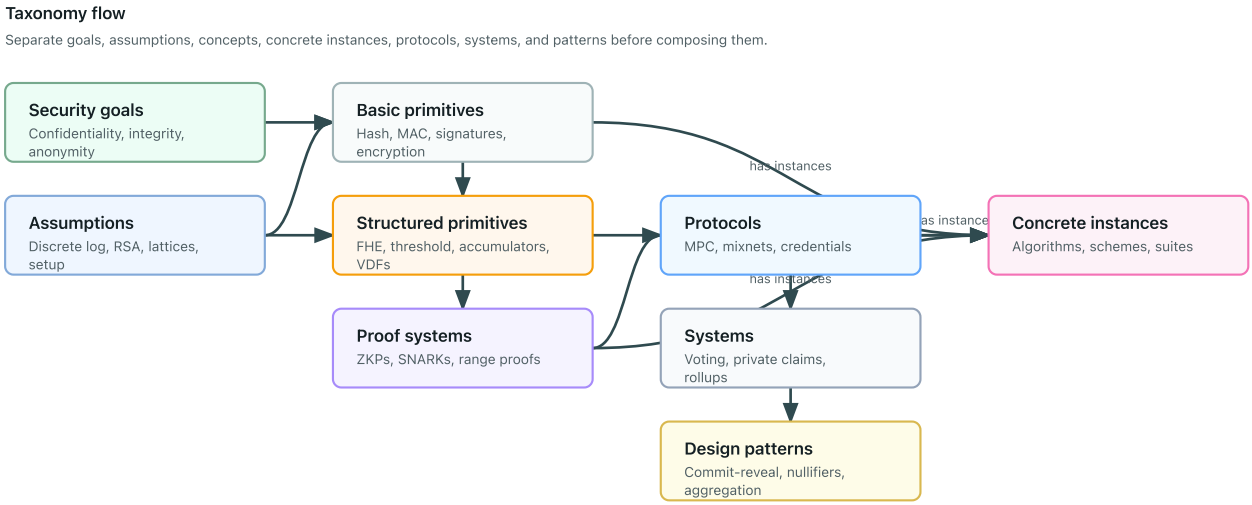
Taxonomy

Taxonomy Overview

The atlas organizes cryptographic ideas from desired outcomes to system patterns:

```
Security goals -> assumptions -> primitives -> structured primitives -> proof systems ->
protocols -> systems -> design patterns
```

Named algorithms, concrete schemes, parameter sets, curves, and protocol suites sit as an instance layer under the concept they instantiate.



This ordering prevents a common mistake: starting with a fashionable tool before stating the problem, the adversary, and the assumptions.

Taxonomy table

Level	Question it answers	Examples
Security goal	What property do we want?	confidentiality, integrity, anonymity, unlinkability
Assumption or substrate	What must be hard, trusted, or available?	discrete logarithm, lattices, pairings, random oracle model
Basic primitive	What small building block provides a narrow guarantee?	hash function, commitment, digital signature
Structured primitive	What extra algebraic or access structure is useful?	homomorphic encryption, threshold cryptography, VDF
Proof system	How can a party prove a statement without revealing too much?	zero-knowledge proof, range proof, membership proof
Protocol	How do parties interact to achieve a goal?	MPC, secure aggregation, mixnet
System or application	What user-facing application combines many tools?	e-voting, private DAO voting, anonymous airdrop

Level	Question it answers	Examples
Design pattern	What reusable composition pattern appears across systems?	anonymous membership, delayed reveal, anti-double-use nullifiers

Concrete instances

A concrete instance is a named algorithm, scheme, parameter family, curve, proof-system construction, or protocol suite that realizes a broader concept.

Instance	Instantiates	Why this placement matters
SHA-256	Hash functions	The page for hash functions explains the general goal; the instance adds concrete deployment and parameter details.
Ed25519	Digital signatures	The scheme inherits the signature concept but has specific curve, encoding, and implementation rules.
ML-KEM	Key encapsulation mechanisms	The instance changes post-quantum posture and parameter choices.
Groth16	Proof systems / SNARKs	The instance has specific trusted-setup and pairing assumptions.
TLS 1.3	Secure-channel protocols	The suite combines primitives, key exchange, authentication, and transcript binding.

Use the parent concept page to explain the security goal, non-goals, assumptions, and failure modes. Use an instance entry or comparison table when the concrete name changes assumptions, maturity, deployment risk, parameter sensitivity, or common misuse patterns. See Concrete Algorithms and Schemes.

Cross-cutting classifications

Two classifications cut across the taxonomy:

- Post-quantum posture: whether a construction is vulnerable, plausibly post-quantum, dependent on instantiation, unknown, or not applicable.
- Confidence models: where confidence comes from, such as public verification, one honest party, `t-of-n` threshold assumptions, honest majority, non-collusion, or a trusted issuer.

Why the levels matter

Each level has different failure modes. A primitive can be mathematically sound and still be used in a protocol that leaks metadata. A protocol can satisfy a narrow model and still fail inside a product because enrollment, timing, recovery, or coercion was ignored.

Practical workflow

When evaluating a design, ask:

1. What security goals are claimed?
2. What assumptions make those goals plausible?

3. Which primitives and protocols provide the claimed properties?
4. Which properties are missing?
5. What happens when the components are composed?

Further reading

- Boneh and Shoup, "A Graduate Course in Applied Cryptography."
- Katz and Lindell, "Introduction to Modern Cryptography."

Security Goals

Security goals are the properties a system is trying to achieve. They should be stated before choosing primitives.

Common goals

Goal	Meaning	Common confusion
Confidentiality	Data is not disclosed to unauthorized parties	Not the same as anonymity
Integrity	Data cannot be modified undetectably	Not the same as authenticity
Authenticity	A message or action is tied to an authorized actor	Not the same as privacy
Anonymity	A subject is hidden within a set	Depends heavily on the size and behavior of the set
Unlinkability	Two actions cannot be linked to the same actor	Does not remove all metadata
Receipt-freeness	A user cannot prove how they acted	Stronger than ballot secrecy
Verifiability	A verifier can check that a claim, proof, tally, or transcript satisfies stated rules	Not the same as truth of off-chain inputs
Auditability	Enough evidence exists to review a process after the fact	Can conflict with privacy if logs are over-collected
Accountability	Misbehavior can be attributed under stated rules	Not the same as public identity disclosure
Forward secrecy	Compromise of a long-term key does not expose past session secrets	Requires protocol-level key evolution
Deniability	A transcript does not convince outsiders who participated or what they said	Can conflict with public verifiability

Why precision matters

Saying "private" is usually too vague. A system may hide values but expose identities, or hide identities but reveal timing. State the goal and the leak separately.

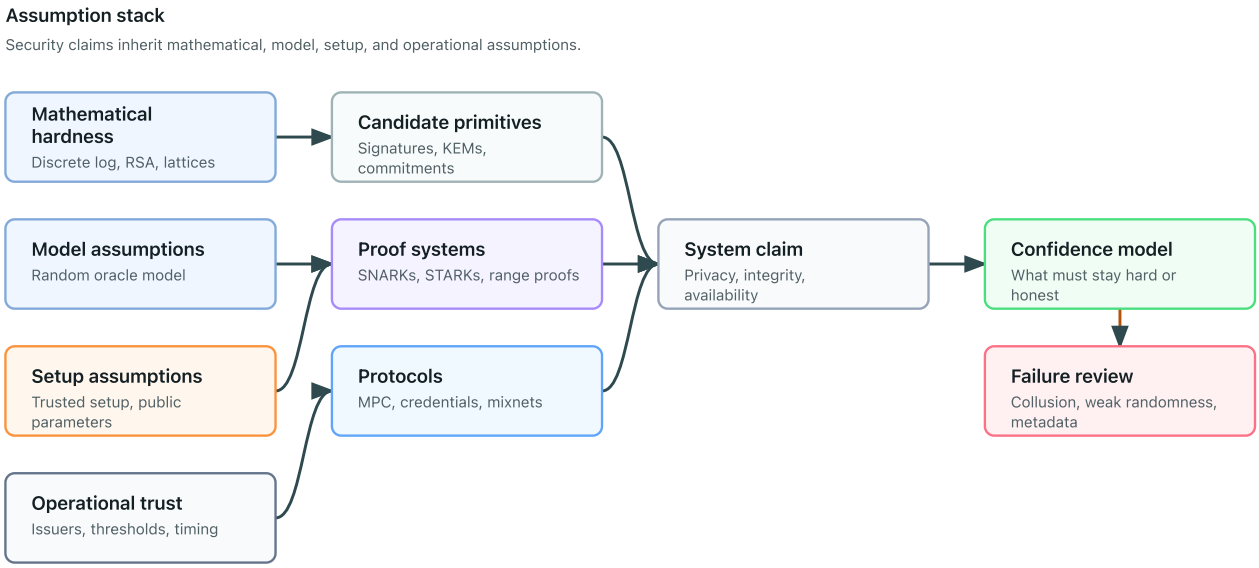
Some goals are primitive-level, such as confidentiality or integrity for a specific message. Others are protocol or system-level, such as coercion resistance, auditability, or forward secrecy. Do not assign a system-level goal to a primitive unless the surrounding protocol assumptions are also stated.

Further reading

- Katz and Lindell, "Introduction to Modern Cryptography."
- Boneh and Shoup, "A Graduate Course in Applied Cryptography."

Assumptions

Cryptographic guarantees are conditional. An assumption is something that must hold for a claim to be meaningful.



Types of assumptions

Type	Examples
Mathematical	discrete logarithm hardness, lattice assumptions
Model	random oracle model, algebraic group model
Setup	trusted setup, common reference string, public parameters
Trust	honest majority, non-collusion, threshold of honest parties
Network	reliable broadcast, authenticated channels, timing bounds
Implementation	secure randomness, constant-time code, safe key storage

Post-quantum posture

Post-quantum posture is a classification of assumptions under a quantum adversary. For example, discrete-logarithm assumptions are quantum-vulnerable, while some hash-based and lattice-based constructions are commonly treated as plausible post-quantum candidates when parameters and implementations are appropriate.

See Post-Quantum Posture.

Confidence model

A confidence model states who or what must remain honest, independent, hard, available, or verifiable. Examples include one-honest-party mixnets, `t-of-n` threshold trustees, honest-majority MPC, public-

verifiable proof systems, trusted issuers, and transparent setup.

See Confidence Models.

Practical checklist

- Who can break the system if they collude?
- What setup must be generated correctly?
- What happens if randomness is weak?
- What metadata remains public?
- Which assumptions are mature and which are research-stage?

Assumption pages

- Discrete logarithm
- Factoring and RSA
- Pairings
- Lattices
- Random oracle model
- Trusted setup

Further reading

- Boneh and Shoup, "A Graduate Course in Applied Cryptography."
- Shor, "Algorithms for Quantum Computation; Discrete Logarithms and Factoring."

Primitives vs Protocols

A primitive is a small building block with a narrow interface. A protocol is a multi-step construction, often involving several parties and several primitives.

A concrete algorithm, scheme, parameter family, or protocol suite is an instance of a concept. It should point back to the parent concept rather than replace it.

Comparison

Aspect	Primitive	Protocol
Scope	Narrow operation	End-to-end interaction
Examples	hash, signature, commitment	MPC, mixnet, anonymous credential issuance
Main question	What property does this operation provide?	What happens across participants, messages, and failure cases?
Common risk	Misstating the guarantee	Ignoring metadata, aborts, or trust assumptions

Where named schemes fit

Named constructions belong under the concept they instantiate:

Named item	Parent concept	Notes
SHA-256	Hash functions	A concrete hash algorithm, not the whole concept of hashing.
Ed25519	Digital signatures	A concrete signature scheme with specific curve and encoding choices.
Groth16	SNARKs / proof systems	A concrete proof system with trusted-setup and pairing assumptions.
TLS 1.3	Secure-channel protocol suite	A concrete protocol suite composed from several primitives and transcript rules.

This keeps the taxonomy concept-first while still letting the book discuss concrete deployment names where they matter.

Example

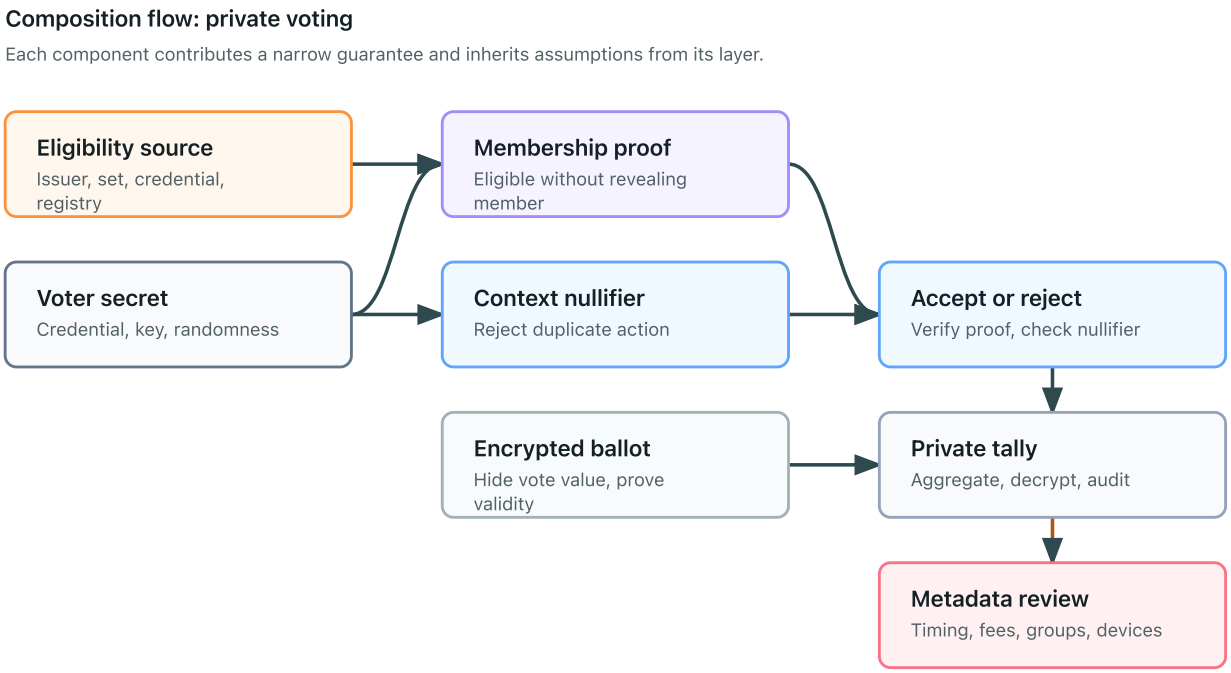
A commitment can bind a value. A voting protocol may use commitments, signatures, zero-knowledge proofs, nullifiers, and tallying rules to achieve a larger goal.

Further reading

- Boneh and Shoup, "A Graduate Course in Applied Cryptography."
- Katz and Lindell, "Introduction to Modern Cryptography."

Composability

Cryptographic tools do not compose automatically. Each primitive or protocol comes with a specific interface, adversary model, setup assumption, and leakage profile.



One-sentence intuition

A component can keep its promise and still leave the larger system insecure if the system needs a different promise.

Common composition mistakes

Mistake	Why it fails
Encryption does not prove validity	A ciphertext can hide an invalid value unless the system also proves the plaintext is well formed.
Commitments do not authenticate users	A commitment binds someone to a value, but it does not say who that someone is.
Zero-knowledge proofs do not hide metadata	A proof can hide a witness while network timing, account history, or reused identifiers remain linkable.
Threshold cryptography is not trustlessness	Threshold schemes replace one trusted party with assumptions about a group, quorum, setup, and key management.
Anonymous credentials do not automatically prevent coercion	A user may still be pressured to reveal secrets, show a transcript, or vote under observation.

What to check

- Are all security goals stated separately?
- Does each component provide exactly the property being claimed?
- Are identities, timestamps, network addresses, and repeated identifiers handled?
- Does setup require a trusted party, ceremony, or honest majority?
- Are invalid inputs rejected without exposing private values?
- Can a malicious participant force aborts, denial of service, or selective failures?

Failure pattern

A system often fails at the boundary between components. For example, a private voting design may encrypt ballots but forget to prove that each encrypted ballot encodes a valid choice. The tally remains private, but a malicious voter may submit malformed data.

Safer framing

Instead of saying "this system is secure," say:

> Under these assumptions, against this adversary, this component contributes this property, while these leaks and non-goals remain.

Further reading

- Canetti, "Universally Composable Security; A New Paradigm for Cryptographic Protocols."
- Katz and Lindell, "Introduction to Modern Cryptography."

Assumptions and Substrates

Discrete Logarithm

One-sentence intuition

The discrete logarithm assumption says that exponentiation in certain finite groups is easy, but reversing it is computationally hard.

Where it sits in the taxonomy

- Level: mathematical assumption or substrate
- Parent category: assumptions
- Related concepts: digital signatures, Pedersen commitments, pairings, key exchange

Problem it solves

Discrete-logarithm hardness gives cryptographic schemes a one-way structure: a secret exponent can create public values that are easy to check or combine but hard to invert.

Mental model

Think of repeatedly stepping around a huge clock. If someone tells you the start point, the step rule, and the final point, finding the exact number of steps should be infeasible in the chosen group.

Minimal example

In a group generated by g, a public value may be written as:

$$Y = g^x$$

The secret is x. The discrete logarithm problem is to recover x from g and Y.

Security properties

- Supports one-way public-key structure in suitable groups.
- Underlies Diffie-Hellman-style key agreement and Schnorr-style signatures.
- Enables binding in many discrete-logarithm-based commitment schemes.

What it does not provide

- Security in every group.
- Protection against quantum computers.
- Safety against bad parameters, small subgroups, or nonce reuse.

Assumptions

The group must be chosen so that known classical attacks are infeasible at the target security level. Implementations must also validate group elements and avoid leaking or reusing secret nonces.

Post-quantum posture

Vulnerable. Shor's algorithm breaks discrete logarithms on a sufficiently large cryptographically relevant quantum computer.

Confidence model

Confidence comes from mathematical-assumption hardness, public parameter review, implementation correctness, and side-channel resistance.

Common constructions

- Diffie-Hellman and elliptic-curve Diffie-Hellman.
- Schnorr, ECDSA, and EdDSA-style signatures.
- Pedersen commitments.
- Pairing-based constructions, with additional assumptions.

Concrete groups and schemes

Concrete family	Common examples	Where it appears	Key differences and cautions
Prime-order elliptic-curve groups	Curve25519, Curve448, P-256, P-384	ECDH, EdDSA, ECDSA, Schnorr-style signatures	Quantum-vulnerable; subgroup and encoding rules are curve-specific.
Blockchain curves	secp256k1	Blockchain signatures and key recovery conventions	Quantum-vulnerable; ecosystem conventions are not interchangeable with other ECDSA settings.
Finite-field groups	FFDHE safe-prime groups	Diffie-Hellman compatibility and standards profiles	Quantum-vulnerable; use reviewed groups and validate parameters.
Discrete-log commitments	Pedersen commitments, inner-product commitments	Confidential values, range proofs, vector commitments	Binding relies on discrete-logarithm hardness and generator setup.
Discrete-log proof systems	Schnorr identification, Bulletproofs	Authentication proofs and range proofs	Fiat-Shamir transcript binding and nonce handling are critical.

Use cases

- Key agreement.
- Digital signatures.
- Commitments and range proofs.
- Pairing-based proof systems.

Composition patterns

Discrete-logarithm assumptions are often paired with transcript hashing, domain separation, nonces, and zero-knowledge proofs. The surrounding protocol must bind public keys and contexts explicitly.

Failure modes and anti-patterns

- Using weak groups or parameters.
- Failing subgroup checks.
- Reusing signature nonces.
- Treating a discrete-logarithm-based system as post-quantum.

Maturity and deployment

Mature and widely deployed, but quantum-vulnerable.

Related concepts

- Digital signatures
- Pedersen commitments
- Key encapsulation and exchange

Further reading

- Diffie and Hellman, "New Directions in Cryptography."
- Shor, "Algorithms for Quantum Computation; Discrete Logarithms and Factoring."
- Boneh and Shoup, "A Graduate Course in Applied Cryptography."

Factoring and RSA

One-sentence intuition

RSA-style cryptography relies on arithmetic modulo a large composite number whose prime factors are secret.

Where it sits in the taxonomy

- Level: mathematical assumption or substrate
- Parent category: assumptions
- Related concepts: public-key encryption, digital signatures, accumulators, VDFs

Problem it solves

RSA gives a public-key trapdoor structure: public operations can be easy while the corresponding private operation depends on knowing the factorization of the modulus.

Mental model

Multiplying two large primes is easy. Recovering those primes from only their product is believed to be hard for classical computers when the primes are generated correctly and are large enough.

Minimal example

An RSA modulus is:

$$N = pq$$

The factors p and q are secret. Many RSA-based systems rely on the difficulty of recovering them from N.

Security properties

- Supports trapdoor public-key encryption and signatures when used in safe schemes.
- Supports RSA accumulators and some time-delay constructions under additional assumptions.

What it does not provide

- Security for textbook RSA.
- Protection against quantum computers.
- Safety if primes, padding, or key sizes are wrong.

Assumptions

RSA systems assume correct prime generation, adequate modulus sizes, safe padding or signature schemes, and protection of the private exponent or factorization.

Post-quantum posture

Vulnerable. Shor's algorithm breaks integer factoring on a sufficiently large cryptographically relevant quantum computer.

Confidence model

Confidence comes from mathematical-assumption hardness, parameter generation, padding or scheme design, and private-key protection.

Common constructions

- RSA encryption schemes.
- RSA signature schemes.
- RSA accumulators.
- Groups of unknown order used in some VDF or timelock designs.

Concrete schemes and parameter families

Scheme or substrate	Common role	Key differences and cautions
RSA-OAEP	Public-key encryption and hybrid encryption	Safe padding is part of the scheme; textbook RSA is not safe encryption.
RSA-PSS	Digital signatures	Preferable to legacy deterministic RSA signatures when RSA signatures are required.
RSA PKCS #1 v1.5 encryption/signatures	Legacy compatibility	Historically fragile; should be treated as legacy context, not a modern design target.
RSA accumulators	Compact membership witnesses	Require an RSA modulus with a clear trust story; setup and update rules dominate confidence.
Unknown-order groups for delay	Repeated-squaring timelocks and VDFs	May use RSA groups or class groups; factorization or setup assumptions must be explicit.
RSA modulus sizes	2048-bit, 3072-bit, 4096-bit profiles	Classical security margin changes with size; all remain quantum-vulnerable.

Use cases

- Legacy public-key encryption and signatures.
- Accumulators.
- Some delay and timelock constructions.

Composition patterns

RSA is commonly used inside hybrid encryption, certificate systems, signatures, and accumulator protocols. Each use needs its own security notion and padding or proof layer.

Failure modes and anti-patterns

- Using textbook RSA directly.
- Reusing primes or generating weak primes.
- Confusing factoring hardness with security of every RSA-based scheme.
- Treating RSA-based systems as post-quantum.

Maturity and deployment

Mature and widely deployed historically, but quantum-vulnerable and being migrated away from in long-term security designs.

Related concepts

- Public-key encryption
- Digital signatures
- Accumulators and Merkle trees

Further reading

- Rivest, Shamir, and Adleman, "A Method for Obtaining Digital Signatures and Public-Key Cryptosystems."
- Shor, "Algorithms for Quantum Computation; Discrete Logarithms and Factoring."
- Boneh and Shoup, "A Graduate Course in Applied Cryptography."

Pairings

One-sentence intuition

A pairing is a special map between algebraic groups that lets hidden exponent relationships become publicly checkable.

Where it sits in the taxonomy

- Level: mathematical assumption or substrate
- Parent category: assumptions
- Related concepts: SNARKs, anonymous credentials, signatures, trusted setup

Problem it solves

Pairings enable compact checks of multiplicative relationships in exponents. This is useful for short signatures, identity-based encryption, polynomial commitments, and many succinct proof systems.

Mental model

A pairing acts like a structured calculator for exponent relationships: it can check that two independently formed group elements contain matching hidden exponent products without revealing those exponents.

Minimal example

At a high level, a bilinear pairing has a relation like:

$$e(g^a, h^b) = e(g, h)^{ab}$$

This bilinearity is powerful, but it comes with specialized assumptions and parameter choices.

Security properties

- Supports compact verification of algebraic relations.
- Enables succinct proof systems and short signature schemes under pairing-specific assumptions.

What it does not provide

- Post-quantum security.
- General-purpose privacy.
- Safety without careful curve and subgroup choices.

Assumptions

Pairing-based systems rely on elliptic-curve group assumptions, pairing-specific hardness assumptions, subgroup checks, and sometimes trusted setup or structured reference strings.

Post-quantum posture

Vulnerable. Common pairings are built from elliptic-curve groups whose discrete logarithm problems are broken by Shor's algorithm.

Confidence model

Confidence comes from mathematical-assumption hardness, curve selection, subgroup validation, setup correctness where required, and careful implementation.

Common constructions

- Pairing-based SNARKs.
- BLS-style signatures.
- Identity-based encryption.
- Polynomial commitments.

Concrete curves and schemes

Family or scheme	Common examples	Where it appears	Key differences and cautions
Pairing-friendly curves	BLS12-381, BN254	SNARKs, BLS signatures, KZG commitments	Quantum-vulnerable; curve security level and subgroup checks are not interchangeable.
BLS signatures	BLS signature variants over pairing-friendly curves	Aggregatable signatures, threshold signatures	Compact aggregation, but domain separation and rogue-key defenses matter.
KZG commitments	KZG polynomial commitments	Rollups, data availability, polynomial openings	Very compact openings; relies on pairings and usually a structured reference string.
Groth16-style SNARKs	Pairing-based succinct proofs	ZK circuits with very small proofs	Often circuit-specific trusted setup; posture inherits pairing vulnerability.
Identity-based encryption	Boneh-Franklin-style systems	Specialized key-management models	Key generator trust is central, not an implementation detail.

Use cases

- Succinct proof verification.
- Aggregatable signatures.
- Anonymous credential schemes.
- Verifiable shuffles and commitments in specialized protocols.

Composition patterns

Pairings are often composed with trusted setup, commitments, zero-knowledge proof statements, and public verification. The setup and subgroup model must be part of the protocol description.

Failure modes and anti-patterns

- Missing subgroup checks.
- Using weak or obsolete curves.
- Hiding trusted setup assumptions behind a "succinct proof" label.
- Treating pairing-based succinctness as post-quantum.

Maturity and deployment

Mature but specialized. Pairing-based systems are deployed, but they require expert parameter and implementation review.

Related concepts

- SNARKs, STARKs, and Bulletproofs
- Anonymous credentials
- Trusted setup

Further reading

- Boneh and Franklin, "Identity-Based Encryption from the Weil Pairing."
- Groth, "On the Size of Pairing-Based Non-interactive Arguments."
- Shor, "Algorithms for Quantum Computation; Discrete Logarithms and Factoring."

Lattices

One-sentence intuition

Lattice-based cryptography relies on the apparent hardness of finding or distinguishing structured noisy points in high-dimensional grids.

Where it sits in the taxonomy

- Level: mathematical assumption or substrate
- Parent category: assumptions
- Related concepts: post-quantum cryptography, homomorphic encryption, key encapsulation, signatures

Problem it solves

Lattice assumptions provide a foundation for many post-quantum candidates and for advanced tools such as fully homomorphic encryption.

Mental model

Imagine a high-dimensional grid where noise slightly moves points away from clean grid locations. Some problems ask an adversary to recover hidden structure from those noisy samples.

Minimal example

Learning with errors (LWE)-style problems involve noisy linear equations. Informally:

$$b = As + e$$

The secret is s, and e is small noise. Recovering s or distinguishing these samples from random should be hard for suitable parameters.

Security properties

- Supports post-quantum key encapsulation and signatures in standardized families.
- Supports homomorphic encryption in many modern schemes.
- Can provide worst-case to average-case style security reductions in some settings.

What it does not provide

- Automatic security for every parameter set.
- Simple implementations.
- Metadata privacy or access control by itself.

Assumptions

Security depends on the concrete lattice problem, dimension, modulus, noise distribution, reduction quality, and implementation side-channel posture.

Post-quantum posture

Plausible. Lattice-based schemes are central post-quantum candidates, but each scheme and parameter set still needs concrete analysis.

Confidence model

Confidence comes from mathematical-assumption hardness, parameter selection, standardization review, and implementation discipline.

Common constructions

- Learning with errors and module-LWE schemes.
- Lattice-based key encapsulation.
- Lattice-based signatures.
- Fully homomorphic encryption schemes.

Concrete assumptions and schemes

Family or scheme	Common role	Key differences and cautions
LWE and module-LWE	Foundation for KEMs and encryption	Parameter choices control concrete security and failure behavior.
SIS and module-SIS	Foundation for signatures and commitments	Often appears in lattice signatures and proof systems.
NTRU-style lattices	Key encapsulation and encryption families	Different structure and parameter trade-offs from module-LWE families.
ML-KEM	Standardized post-quantum key encapsulation	Plausibly post-quantum; larger public keys and ciphertexts affect protocols.
ML-DSA	Standardized post-quantum signatures	Plausibly post-quantum; signature size and deterministic/randomized signing choices matter.
BFV, BGV, CKKS, TFHE/FHEW	Homomorphic encryption families	Workload fit, noise growth, bootstrapping, and approximation behavior differ sharply.

Use cases

- Post-quantum key establishment.
- Post-quantum signatures.
- Homomorphic encryption.
- Some functional-encryption research.

Composition patterns

Lattice-based components are often combined with symmetric encryption, KDFs, signatures, and protocol transcript binding. Migration designs may combine classical and post-quantum mechanisms during transition.

Failure modes and anti-patterns

- Choosing ad hoc parameters.
- Ignoring decryption-failure behavior.
- Treating all lattice assumptions as interchangeable.
- Assuming post-quantum posture removes implementation risk.

Maturity and deployment

Emerging to deployed depending on the scheme. ML-KEM and ML-DSA are standardized, while many advanced lattice tools remain specialized or research-stage.

Related concepts

- Post-quantum posture
- Homomorphic encryption
- Key encapsulation and exchange

Further reading

- Regev, "On Lattices, Learning with Errors, Random Linear Codes, and Cryptography."
- NIST FIPS 203, "Module-Lattice-Based Key-Encapsulation Mechanism Standard."
- Gentry, "Fully Homomorphic Encryption Using Ideal Lattices."

Random Oracle Model

One-sentence intuition

The random oracle model treats a hash function as an ideal public random function inside a security proof.

Where it sits in the taxonomy

- Level: mathematical assumption or model
- Parent category: assumptions
- Related concepts: hash functions, Fiat-Shamir transforms, zero-knowledge proofs

Problem it solves

The model lets designers prove security for efficient protocols that use hash functions as if every new query returned an independent random value.

Mental model

Imagine a public notebook that answers every new input with a fresh random-looking output and always repeats the same answer for the same input.

Minimal example

A protocol proof may analyze calls to an ideal oracle:

$$y \leftarrow \quad (x)$$

In implementation, the oracle is replaced with a concrete hash function such as SHA-2, SHA-3, or a proof-system-specific hash.

Security properties

- Supports security arguments for many practical hash-based transforms.
- Helps model Fiat-Shamir-style challenge generation.

What it does not provide

- A guarantee that any concrete hash exactly behaves like a random oracle.
- Protection against bad domain separation.
- A substitute for checking the implemented statement or protocol context.

Assumptions

The proof assumes ideal random-oracle behavior. The implementation assumes the chosen hash function and domain separation are good enough for the intended security target.

Post-quantum posture

Not applicable to the model itself. Concrete hash functions can be plausibly post-quantum with adequate output lengths, while the protocol's surrounding assumptions may not be.

Confidence model

Confidence comes from the proof model, conservative hash selection, domain separation, and awareness that the model is idealized.

Common constructions

- Fiat-Shamir transforms.
- Hash-and-sign style designs.
- Non-interactive proof systems.
- Hash-based commitments and nullifiers.

Concrete instantiations

Concrete choice	Typical role	Key differences and cautions
SHA-256 / SHA-512	General random-oracle-like hashing in protocols	Conservative deployed choices, but domain labels and transcript encodings are still required.
SHA3 and SHAKE	Random-oracle-like hashing and extendable output	Variable output from SHAKE must be treated as a parameter, not an afterthought.
Hash-to-curve suites	Mapping arbitrary strings into elliptic-curve groups	Needs standardized encodings; ad hoc mappings can bias outputs or break proofs.
Poseidon / Rescue / MiMC	Proof-system-specific hashes in circuits	Chosen for circuit efficiency; the security argument is tied to the proof-system context.
Fiat-Shamir transcripts	Challenge derivation for non-interactive proofs	Transcript encoding must bind statements, public inputs, protocol version, and domain.

Use cases

- Security proofs for practical protocols.
- Challenge derivation.
- Transcript binding.

Composition patterns

Random-oracle-model arguments often appear with zero-knowledge proofs, signatures, commitments, and protocol transcripts. The same hash function should be domain-separated across roles.

Failure modes and anti-patterns

- Treating a random-oracle proof as an implementation proof for every hash.
- Reusing transcript encodings ambiguously.
- Omitting domain labels.
- Confusing the model with a concrete primitive.

Maturity and deployment

Mature as a proof model, but idealized. It should be named explicitly when a construction relies on it.

Related concepts

- Hash functions
- Zero-knowledge proofs
- Commitments

Further reading

- Bellare and Rogaway, "Random Oracles are Practical; A Paradigm for Designing Efficient Protocols."
- Boneh and Shoup, "A Graduate Course in Applied Cryptography."

Trusted Setup

One-sentence intuition

Trusted setup is a parameter-generation process whose hidden secrets must be destroyed or never known for the resulting system to be sound or private.

Where it sits in the taxonomy

- Level: setup or trust assumption
- Parent category: assumptions
- Related concepts: SNARKs, pairings, common reference strings, public verifiability

Problem it solves

Some efficient proof systems and commitments need structured public parameters. A setup process creates those parameters, but the security model must say what hidden setup material could break the system if retained.

Mental model

Think of forging a lock with a temporary master key. The lock can be public, but the temporary master key must be destroyed; otherwise whoever keeps it may forge openings or proofs.

Minimal example

A setup ceremony may output public parameters:

$$\mathrm{pp} \leftarrow \mathrm{Setup}(\lambda)$$

The dangerous part is any hidden trapdoor or toxic waste used to create pp.

Security properties

- Enables compact or efficient proof systems in some families.
- Can support public verification if setup assumptions hold.

What it does not provide

- Trustlessness.
- Protection if all ceremony participants collude or the trapdoor remains.
- A guarantee that the statement being proven is correct.

Assumptions

The setup ceremony, transcript, randomness contributions, and software environment must match the construction's assumptions. Multi-party ceremonies often rely on at least one honest participant destroying their secret contribution.

Post-quantum posture

Depends on the construction. Many trusted-setup systems are pairing-based and quantum-vulnerable; the setup model itself is a trust assumption rather than a post-quantum primitive.

Confidence model

Confidence comes from trusted-setup or one-honest-party ceremony assumptions, transcript auditability, implementation review, and public verification of parameters.

Common constructions

- Structured reference strings.
- Powers-of-tau style ceremonies.
- Circuit-specific or universal SNARK setup.
- Multi-party parameter-generation ceremonies.

Concrete setup patterns

Setup pattern	Common examples	Key differences and cautions
Circuit-specific setup	Groth16-style circuits	Small proofs, but every circuit may need its own setup assumptions.
Universal structured setup	Powers-of-tau, PLONK-style SRS reuse	Can support many circuits up to parameter limits; toxic waste and transcript audit remain central.
Transparent setup	STARK-style systems, many Bulletproof-style systems	Avoids trusted toxic waste, but still has parameters and implementation assumptions.
Updatable ceremonies	Multi-party SRS updates	Often rely on at least one honest contribution; update verification must be public.
Application-specific parameters	KZG commitments, accumulator parameters	Reusing parameters outside their intended scope can invalidate the confidence model.

Use cases

- Pairing-based SNARKs.
- Polynomial commitments.
- Some anonymous credential or accumulator systems.

Composition patterns

Trusted setup is commonly combined with public verifiability: anyone can verify proofs after setup, but only if the setup assumptions are accepted.

Failure modes and anti-patterns

- Hiding the setup assumption from users.
- Losing ceremony transcripts.
- Treating "multi-party setup" as equivalent to no setup.
- Reusing parameters outside their intended scope.

Maturity and deployment

Mature but specialized. Setup ceremonies are deployed in some proof-system ecosystems, but they remain a major confidence-model component.

Related concepts

- SNARKs, STARKs, and Bulletproofs
- Pairings
- Confidence models

Further reading

- Groth, "On the Size of Pairing-Based Non-interactive Arguments."
- Bowe, Gabizon, and Miers, "Scalable Multi-party Computation for zk-SNARK Parameters in the Random Beacon Model."

Basic Primitives

Basic Primitives Overview

Basic primitives provide narrow cryptographic guarantees. They are not complete systems.

Examples

- Hash functions compress data into fixed-length digests.
- Symmetric encryption protects bulk data under a shared secret key.
- Authenticated encryption combines confidentiality with ciphertext integrity and authenticated public context.
- Message authentication codes authenticate messages with a shared secret key.
- Key derivation functions separate keys by context.
- Randomness and nonces provide freshness or uniqueness where schemes require it.
- Commitments bind a party to a hidden value.
- Digital signatures authenticate messages.
- Public-key encryption lets a sender encrypt to a recipient's public key.
- Key encapsulation and exchange establish shared secret material.
- Secret sharing splits a secret across multiple shares.

Reading rule

For each primitive, ask what it guarantees, what it assumes, what it does not provide, and what breaks when it is composed incorrectly.

Coverage note

This section now covers the main symmetric-key and public-key primitives that most applied systems rely on. Remaining primitive-level gaps are tracked in Editorial Maturity and Coverage.

Hash Functions

One-sentence intuition

A cryptographic hash function maps data to a fixed-length digest in a way that should resist finding collisions or reversing the digest.

Security properties

- Preimage resistance.
- Second-preimage resistance.
- Collision resistance.

What it does not provide

- Encryption.
- Authentication unless used in a construction such as a MAC.
- Hiding for low-entropy secrets without careful salting or keying.

Use cases

- Commitments.
- Merkle trees.
- Content addressing.
- Transcript binding in protocols.

Concrete algorithms and schemes

Family	Examples	Where readers usually see it	Key differences and cautions
SHA-2	SHA-256, SHA-384, SHA-512	General-purpose hashing, signatures, Merkle trees, protocol transcripts	Conservative deployed default; choose output length for the security target and domain-separate protocol roles.
SHA-3 and SHAKE	SHA3-256, SHA3-512, SHAKE128, SHAKE256	Hashing, extendable-output functions, post-quantum schemes	Sponge-based design; SHAKE outputs variable length, so the output length is a security parameter.
BLAKE family	BLAKE2, BLAKE3	File integrity, application protocols, high-throughput hashing	Fast and widely used in software; check whether the exact variant is standardized or ecosystem-specific.
ZK-friendly hashes	Poseidon, Rescue, MiMC, Griffin	Zero-knowledge circuits and proof systems	Optimized for arithmetic circuits, not a drop-in replacement for general-purpose hashing unless the full protocol expects that choice.
Legacy or broken hashes	SHA-1, MD5	Old protocols, compatibility checks, forensic context	Collision resistance is broken or deprecated; include only to explain legacy risk, not for new designs.

Assumptions

The chosen hash function must be within its intended security lifetime, outputs must be long enough for the security target, and protocols must use clear domain separation when the same function is reused in different roles.

Post-quantum posture

Plausible with appropriate output lengths and parameters. Hash functions are often used as post-quantum building blocks, but the security target must account for quantum search speedups.

Confidence model

Confidence comes from public algorithm scrutiny, parameter choice, domain separation, and correct use. Hashing a low-entropy secret is not enough to make it hidden.

Failure modes and anti-patterns

- Hashing passwords without a password-hashing scheme.
- Treating a hash of a small secret as hidden.
- Using obsolete or broken hash functions.

Further reading

- Boneh and Shoup, "A Graduate Course in Applied Cryptography."
- Grover, "A Fast Quantum Mechanical Algorithm for Database Search."
- NIST FIPS 180-4, "Secure Hash Standard."
- NIST FIPS 202, "SHA-3 Standard."

Symmetric Encryption

One-sentence intuition

Symmetric encryption uses the same secret key to encrypt and decrypt data.

Security properties

- Confidentiality under the chosen attack model.
- Efficient protection for bulk data when used through a safe mode or authenticated-encryption construction.

For most new protocol designs, the safer interface is authenticated encryption, not bare encryption.

What it does not provide

- Key agreement or key distribution.
- Authentication unless combined with a message authentication code or authenticated encryption.
- Metadata privacy for sizes, timing, or access patterns.

Assumptions

The key must remain secret, nonces or initialization vectors must follow the scheme rules, and the encryption mode must match the threat model. Raw block ciphers should not be used as complete encryption schemes.

Post-quantum posture

Plausible with appropriate key sizes and conservative parameters. Quantum search affects security margins, so symmetric-key migration often increases key sizes rather than replacing the primitive family.

Confidence model

Confidence comes from secret-key control, public scrutiny of the algorithm, correct nonce handling, and authenticated use when active attackers can modify ciphertexts.

Use cases

- Bulk data encryption.
- Encrypted backups.
- The data-encryption part of hybrid encryption.

Concrete algorithms and schemes

Scheme or mode	Primitive family	Typical role	Key differences and cautions
AES-GCM	AES block cipher plus Galois/Counter Mode	Authenticated encryption for network protocols and storage	Very common and hardware-accelerated; nonce reuse is catastrophic.
ChaCha20-Poly1305	Stream cipher plus MAC	Authenticated encryption in software and mobile environments	Often faster without AES hardware; nonces must still be unique per key.
XChaCha20-Poly1305	Extended-nonce ChaCha20-Poly1305 variant	Applications that want random nonces with a larger nonce space	Useful engineering shape, but check ecosystem support and protocol compatibility.
AES-GCM-SIV / AES-SIV	Misuse-resistant authenticated encryption	Systems where accidental nonce reuse is a realistic risk	More forgiving of nonce mistakes, but not a license to ignore nonce design.
AES-CBC plus MAC	Legacy composition	Older protocols and compatibility layers	Only safe with correct encrypt-then-MAC composition and padding handling; avoid for new designs when AEAD is available.
AES-ECB	Raw block-cipher mode	Legacy anti-pattern	Reveals repeated plaintext blocks and should not be used for protecting structured data.

Failure modes and anti-patterns

- Reusing nonces in modes that require uniqueness.
- Using encryption without authentication.
- Designing a custom mode around a block cipher or stream cipher.

Further reading

- Boneh and Shoup, "A Graduate Course in Applied Cryptography."
- Katz and Lindell, "Introduction to Modern Cryptography."
- NIST FIPS 197, "Advanced Encryption Standard."
- RFC 5116, "An Interface and Algorithms for Authenticated Encryption."
- RFC 8439, "ChaCha20 and Poly1305 for IETF Protocols."

Authenticated Encryption

One-sentence intuition

Authenticated encryption protects a message's confidentiality and lets the receiver reject modified ciphertexts.

Where it sits in the taxonomy

- Level: basic primitive.
- Parent category: symmetric cryptography.
- Related concepts: Symmetric encryption, Message authentication codes, Randomness and nonces.

Problem it solves

Many systems need both secrecy and tamper detection. Encrypting without authentication can allow an attacker to modify ciphertexts and learn from error behavior, while authenticating the plaintext separately can fail if the composition order, keys, or associated context are wrong.

Authenticated encryption gives applications a single interface for "encrypt this plaintext, authenticate this public context, and reject invalid ciphertexts."

Mental model

Think of a sealed envelope with a tamper-evident seal. The inside of the envelope is hidden. The label on the outside may remain visible, but the seal binds that label to the hidden contents so an attacker cannot swap either one without detection.

Minimal example

A protocol sends:

- plaintext: `transfer 10 tokens`;
- associated data: `chain_id=1, protocol=v2, sender=alice`;
- nonce: a value that must be unique for the key.

The receiver decrypts only if the ciphertext, tag, associated data, key, and nonce all verify together. If the associated data changes to `chain_id=2`, verification fails even though the associated data was never encrypted.

Security properties

- Confidentiality of the plaintext under the scheme's chosen security notion.
- Integrity of the ciphertext and plaintext.
- Authenticity under the symmetric key: only someone with the key should produce ciphertexts that verify.

- Associated-data integrity: public context is authenticated even though it is not encrypted.

What it does not provide

- Non-repudiation or public verifiability.
- Sender identity when several parties share the same key.
- Protection from nonce reuse unless the specific scheme is misuse-resistant.
- Hiding of associated data, message length, timing, or traffic patterns.
- Post-compromise protection after the symmetric key is exposed.

Assumptions

The key must remain secret, nonces must satisfy the scheme's exact uniqueness or randomness rule, associated data must include the full protocol context that needs binding, and implementations must reject invalid tags before releasing plaintext.

Post-quantum posture

Plausible when instantiated with symmetric-key algorithms using appropriate key and tag lengths. Quantum search reduces effective brute-force margins, so parameter choices still matter. The surrounding key-establishment or signature layer may be quantum-vulnerable even if the authenticated-encryption algorithm is not.

Confidence model

Confidence comes from the symmetric-key assumption, correct nonce management, unambiguous associated-data encoding, and implementations that do not leak plaintext or tag-check behavior through side channels.

Common constructions

Construction	Typical use	Caution
AES-GCM	Network protocols and hardware-accelerated platforms	Catastrophic nonce reuse under one key.
ChaCha20-Poly1305	Software-oriented secure channels	Nonce uniqueness is still required.
XChaCha20-Poly1305	Systems that want larger random nonces	Widely used, but not every protocol registry includes it.
AES-GCM-SIV / AES-SIV	Misuse-resistant encryption	More forgiving of nonce mistakes, but still has limits and different performance.
AES-CCM	Constrained and wireless protocols	Nonce formatting and length choices are easy to get wrong.

Use cases

- TLS record protection.

- Encrypted application messages.
- File and backup encryption.
- Token sealing.
- Encrypted database fields where context must be bound as associated data.

Composition patterns

Authenticated encryption is commonly fed by a key derivation function, uses randomness and nonces, and appears inside secure-channel protocols. It often binds transcript hashes, version numbers, identities, or domain separators as associated data.

Failure modes and anti-patterns

- Reusing a nonce with AES-GCM or ChaCha20-Poly1305 under the same key.
- Decrypting or parsing plaintext before tag verification succeeds.
- Leaving protocol version, sender, recipient, or purpose outside associated data.
- Mixing encrypted values from different protocols because encodings are not domain-separated.
- Treating authenticated encryption as a signature.
- Using AES-CBC or AES-CTR without a correct authentication layer.

Maturity and deployment

Widely deployed. Authenticated encryption with associated data (AEAD) is the default interface in modern secure-channel and application-message designs, but incorrect nonce handling remains a common implementation risk.

Related concepts

- Symmetric encryption
- Message authentication codes
- Randomness and nonces
- Key derivation functions
- Metadata leakage

Further reading

- RFC 5116, "An Interface and Algorithms for Authenticated Encryption."
- RFC 8439, "ChaCha20 and Poly1305 for IETF Protocols."
- Rogaway, "Authenticated-Encryption with Associated-Data."
- NIST SP 800-38D, "Recommendation for Block Cipher Modes of Operation: Galois/Counter Mode (GCM) and GMAC."

Message Authentication Codes

One-sentence intuition

A message authentication code (MAC) lets parties sharing a secret key detect forged or modified messages.

Security properties

- Message integrity.
- Symmetric-key authenticity between parties that share the key.

What it does not provide

- Public verifiability.
- Non-repudiation.
- Confidentiality.
- Protection if all verifiers share the same signing key and one is compromised.

Assumptions

The MAC key must remain secret, message encoding must be unambiguous, and verifiers must bind the tag to the right protocol context and replay rules.

Post-quantum posture

Plausible for standard MAC constructions with appropriate key and tag sizes. The surrounding key-exchange or provisioning layer may still be quantum-vulnerable.

Confidence model

Confidence comes from symmetric key secrecy, unforgeability of the MAC construction, and replay protection in the surrounding protocol.

Use cases

- Authenticated APIs.
- Protocol transcript authentication.
- Authenticated encryption internals.

When confidentiality and integrity are both required for ciphertexts, prefer a reviewed authenticated-encryption scheme over designing a custom encryption-plus-MAC composition.

Concrete algorithms and schemes

Scheme	Built from	Typical role	Key differences and cautions
HMAC-SHA-256 / HMAC-SHA-512	Hash function	General-purpose MAC and KDF component	Mature and conservative; key separation and unambiguous message encoding still matter.
CMAC-AES	AES block cipher	MACs in AES-centered systems	Avoids naive CBC-MAC pitfalls when used as specified.
GMAC	GCM authentication component	Authentication when AES-GCM infrastructure is already present	Requires nonce discipline; misuse can break authenticity.
Poly1305	One-time MAC, commonly paired with ChaCha20	AEAD internals such as ChaCha20-Poly1305	Requires one-time keys derived correctly; do not reuse Poly1305 keys directly.
KMAC	SHA-3/cSHAKE family	SHA-3-based keyed hashing	Useful where SHA-3 primitives are already part of the design.
CBC-MAC	Block cipher	Narrow legacy setting	Unsafe when copied outside its fixed-length, single-key assumptions.

Failure modes and anti-patterns

- Reusing one MAC key across unrelated protocols without domain separation.
- Comparing tags with timing leaks.
- Treating a MAC as a digital signature that third parties can verify.

Further reading

- Boneh and Shoup, "A Graduate Course in Applied Cryptography."
- Katz and Lindell, "Introduction to Modern Cryptography."
- NIST SP 800-38B, "Recommendation for Block Cipher Modes of Operation: The CMAC Mode for Authentication."

Key Derivation Functions

One-sentence intuition

A key derivation function (KDF) turns shared secret material into context-specific cryptographic keys.

Security properties

- Key separation between contexts.
- Pseudorandom derived keys when the input secret has enough entropy.
- Binding to protocol labels, salts, and transcript context when used correctly.

What it does not provide

- Entropy that was not present in the input.
- Password hardening unless the KDF is specifically designed for passwords.
- Agreement that the input secret is authentic.

Assumptions

The input secret must have adequate entropy for the use case, salts and labels must be chosen correctly, and each derived key must have a clearly separated purpose.

Post-quantum posture

Plausible for hash-based KDFs with appropriate parameters. The posture of the key-establishment step that produced the input secret must be classified separately.

Confidence model

Confidence comes from entropy in the input material, domain separation in the KDF inputs, and disciplined key lifecycle management.

Use cases

- Deriving encryption and MAC keys from a shared secret.
- Session-key schedules.
- Context-separated keys for protocols.

Concrete algorithms and schemes

Scheme	Main input shape	Typical role	Key differences and cautions
HKDF	High-entropy shared secret plus salt and info labels	Session key schedules and protocol key separation	Fast KDF; not a password-hashing scheme.

Scheme	Main input shape	Typical role	Key differences and cautions
PBKDF2	Password plus salt and iteration count	Legacy password-based key derivation	Widely deployed, but easier to accelerate than modern memory-hard schemes.
scrypt	Password plus salt and memory/cost parameters	Password hashing and key derivation	Adds memory cost; parameters must match attacker hardware assumptions.
Argon2id	Password plus salt and memory/time/parallelism parameters	Password storage and password-derived keys	Modern memory-hard default when available; parameters need operational tuning.
NIST counter-mode KDFs	Shared secret plus labels and context	Key management in NIST-profiled systems	Good for structured key derivation when labels and context are explicit.

Failure modes and anti-patterns

- Reusing one derived key for multiple purposes.
- Feeding low-entropy passwords into a fast KDF.
- Omitting transcript or domain labels, which can create cross-protocol key reuse.

Further reading

- Boneh and Shoup, "A Graduate Course in Applied Cryptography."
- Katz and Lindell, "Introduction to Modern Cryptography."
- RFC 5869, "HMAC-based Extract-and-Expand Key Derivation Function."

Randomness and Nonces

One-sentence intuition

Randomness and nonces provide the fresh or unique values that many cryptographic schemes need to stay secure.

Security properties

- Unpredictability for secrets, keys, salts, and some protocol challenges.
- Uniqueness for nonces in schemes that require no reuse.

What it does not provide

- Security if the surrounding scheme is misused.
- Confidentiality or authentication by itself.
- A substitute for domain separation.

Assumptions

Random values must come from an appropriate random-number generator, nonces must satisfy the exact uniqueness or unpredictability rule of the scheme, and failures must be detectable where possible.

Post-quantum posture

Not applicable as a standalone category. Randomness quality matters equally in classical and post-quantum systems.

Confidence model

Confidence depends on local entropy sources, deterministic derivation where appropriate, implementation checks, and operational monitoring for reuse or generator failure.

Use cases

- Encryption nonces.
- Signature nonces.
- Commitment randomness.
- Protocol challenges.

Concrete generators and nonce patterns

Mechanism or pattern	Typical role	Key differences and cautions
Operating-system CSPRNGs	General key, salt, nonce, and challenge generation	Usually the right source for application randomness; failures often come from bypassing it.

Mechanism or pattern	Typical role	Key differences and cautions
Hash_DRBG / HMAC_DRBG / CTR_DRBG	Deterministic random bit generators	Common in standards-oriented libraries and hardware modules; require correct seeding and reseeding.
ChaCha20-based CSPRNGs	Fast software randomness expansion	Common in operating systems and libraries; security depends on seed handling and state protection.
Counter nonces	Unique nonces for one key and one stream of messages	Good when state is reliable; dangerous if state rolls back or keys are reused.
Random nonces	Large nonce spaces where collision probability is negligible	Requires enough nonce bits; small random nonces can collide under load.
Synthetic IV / deterministic nonce designs	Misuse-resistant encryption modes and deterministic signatures	Reduces reliance on external randomness, but only within schemes designed for that model.

Failure modes and anti-patterns

- Reusing a signature nonce in schemes where it exposes the private key.
- Reusing encryption nonces in modes that require uniqueness.
- Treating predictable identifiers as random values.

Further reading

- Boneh and Shoup, "A Graduate Course in Applied Cryptography."
- Katz and Lindell, "Introduction to Modern Cryptography."

Commitments

One-sentence intuition

A commitment lets someone lock in a value now and reveal it later.

Where it sits in the taxonomy

- Level: basic primitive
- Parent category: primitives
- Related concepts: hash functions, Pedersen commitments, zero-knowledge proofs

Problem it solves

Commitments solve the problem of delayed disclosure. A party can choose a value, publish evidence that the value has been fixed, and later open the commitment so others can check that the value did not change.

Mental model

Think of placing a message in a locked box and publishing the box. Later, the sender opens the box. A good commitment hides the message while the box is locked and prevents the sender from opening the same box to a different message.

Minimal example

A simple hash commitment can look like:

$$c = H(m \,\|\, r)$$

The opening is the pair (m, r).

The verifier recomputes the hash during opening and checks that it matches the published commitment.

More abstractly, a commitment scheme has two operations:

$$c \leftarrow \mathrm{Commit}(m; r)$$

$$\mathrm{Open}(c, m, r) \in \{\mathrm{accept}, \mathrm{reject}\}$$

Security properties

- Hiding: the commitment should not reveal the committed value before opening.
- Binding: the committer should not be able to open the same commitment to two different values.

Some schemes are computationally hiding or binding; others are perfectly hiding or binding under their model.

What it does not provide

- Authentication of the committer.
- Proof that the committed value is valid.
- Confidentiality after the opening is revealed.
- Protection against weak randomness.
- Agreement about when openings are allowed.

Assumptions

Hash commitments usually rely on properties of the hash function and sufficient randomness. Pedersen commitments rely on group assumptions and generator choices.

Post-quantum posture

Depends on the construction. Hash-based commitments can be plausibly post-quantum with appropriate hash choices and parameters. Pedersen commitments and other discrete-logarithm-based commitments are quantum-vulnerable.

Confidence model

Confidence comes from the hiding and binding assumptions of the concrete commitment scheme, plus correct randomness and unambiguous opening rules. A commitment does not identify the committer unless authentication is added.

Common constructions

- Hash-based commitments.
- Pedersen commitments.
- Merkle tree commitments to many values.

Concrete algorithms and schemes

Scheme	Typical role	Hiding and binding shape	Key differences and cautions
Hash commitment	Commit to one value with `H(message		randomness)`
Pedersen commitment	Commit to numeric values with additive homomorphism	Perfectly hiding, computationally binding under discrete-logarithm assumptions	Useful for range proofs and confidential values; quantum-vulnerable.

Scheme	Typical role	Hiding and binding shape	Key differences and cautions
Merkle commitment	Commit to a list or set of values	Binding from collision resistance	Efficient membership proofs; privacy depends on leaf encoding and whether paths reveal structure.
KZG commitment	Polynomial or vector commitment	Binding under pairing assumptions and setup model	Very compact openings; quantum-vulnerable and often setup-dependent.
Inner-product-argument commitments	Vector/polynomial commitments in discrete-logarithm groups	Binding under discrete-logarithm assumptions	Avoids trusted setup in common forms, but proofs can be larger and quantum-vulnerable.

Use cases

- Commit-reveal protocols.
- Private voting and delayed ballot reveal.
- Range proofs and confidential transactions.
- Randomness beacons and fair ordering protocols.

Composition patterns

Commitments are often combined with zero-knowledge proofs to prove facts about a hidden value without opening it. They are also used with signatures when a system must bind a commitment to an authenticated party.

Failure modes and anti-patterns

- Reusing or choosing predictable randomness can expose the committed value.
- Forgetting authentication allows anyone to submit a commitment.
- Treating a commitment as encryption confuses delayed disclosure with recipient-controlled confidentiality.
- Opening rules can be abused if the protocol does not define deadlines and abort handling.

Maturity and deployment

Commitments are mature and widely used, but concrete schemes still depend on careful parameter choices and protocol context.

Related concepts

- Pedersen commitments
- Zero-knowledge proofs
- Private aggregation

Further reading

- Boneh and Shoup, "A Graduate Course in Applied Cryptography."
- Pedersen, "Non-Interactive and Information-Theoretic Secure Verifiable Secret Sharing."

Pedersen Commitments

One-sentence intuition

A Pedersen commitment hides a value while preserving additive structure.

Where it sits in the taxonomy

- Level: structured primitive
- Parent category: commitments
- Related concepts: commitments, homomorphic commitments, range proofs

Problem it solves

Pedersen commitments let a system commit to numeric values and later use algebraic relationships between commitments. This is useful when a protocol needs to keep values hidden while proving or aggregating relationships about them.

Mental model

The commitment mixes the message with a private random mask. Observers see a group element that looks randomized, while the committer can later reveal both the value and the mask.

Minimal example

A common notation is:

$$C = g^m h^r$$

Here `m` is the message, `r` is randomness, and `g` and `h` are group generators. The opening is `m, r`.

The additive structure appears when two commitments are combined:

$$C_1 \cdot C_2 = g^{m_1+m_2} h^{r_1+r_2}$$

Inline notation also works: $C = g^m h^r$.

Security properties

- Hiding: if `r` is random and secret, the commitment hides `m`.
- Binding: under discrete logarithm assumptions, a committer should not be able to open one commitment to two different messages.
- Additive homomorphism: multiplying commitments corresponds to adding their committed values.

What it does not provide

- Authentication.
- Proof that `m` is in an allowed range.
- Protection if randomness is reused, predictable, or revealed.
- Binding if the discrete logarithm relationship between `g` and `h` is known to the committer.

Assumptions

Pedersen commitments depend on a suitable group where discrete logarithms are hard and on safe generation of independent generators. The randomness must be sampled correctly and kept secret until opening.

Post-quantum posture

Vulnerable. Standard Pedersen commitments rely on discrete-logarithm hardness, so they should not be treated as post-quantum commitments.

Confidence model

Confidence comes from the group assumption, independent public generator setup, and correct randomness. Binding can fail if a party knows the discrete-logarithm relation between the generators. Hiding can fail if randomness is reused, predictable, or revealed.

Common constructions

Pedersen commitments are usually instantiated in elliptic curve or finite-field groups. Concrete deployments must choose parameters and libraries carefully.

Concrete instantiations

Instantiation	Common setting	Key differences and cautions
Elliptic-curve Pedersen commitments	Confidential transactions, range proofs, ZK protocols	Efficient and common; curve choice, generator derivation, and subgroup handling matter.
Finite-field Pedersen commitments	Older protocols and threshold/verifiable secret sharing	Same discrete-logarithm posture, but parameter sizes and subgroup checks differ.
Pedersen vector commitments	Commit to several values with independent generators	Useful for proving linear relations; binding depends on no party knowing generator relations.
Bulletproof-style commitments	Range proofs and confidential amounts	Often built over Pedersen commitments; range proofs are needed to prevent overflow or negative-value attacks.

Use cases

- Confidential transactions.
- Range proofs.
- Private tallying.
- Commitments inside zero-knowledge proof systems.

Composition patterns

Pedersen commitments are often paired with range proofs to show that hidden values are non-negative or within a bound. They can also support private tallying because commitments to individual votes can be combined into a commitment to the total.

Failure modes and anti-patterns

- Bad randomness can reveal values or allow linkage.
- Reusing randomness across commitments can leak relationships between messages.
- Unknown generator setup can break binding if a party knows the relation between generators.
- Homomorphism can be dangerous if a protocol forgets to constrain values with range proofs.

Maturity and deployment

Pedersen commitments are mature but should be used through well-reviewed libraries and protocol designs.

Related concepts

- Commitments
- Range proofs
- Private aggregation

Further reading

- Pedersen, "Non-Interactive and Information-Theoretic Secure Verifiable Secret Sharing."
- Bünz et al., "Bulletproofs: Short Proofs for Confidential Transactions and More."

Digital Signatures

One-sentence intuition

A digital signature lets a private key holder authorize a message so anyone with the public key can verify it.

Security properties

- Message authenticity.
- Integrity.
- Non-repudiation in some legal or operational contexts, depending on key control.

What it does not provide

- Confidentiality.
- Anonymity.
- Proof that the signer understood the message.
- Protection if the private key is stolen.

Use cases

- Software updates.
- Blockchain transactions.
- Credential issuance.
- Protocol transcript authentication.

Concrete algorithms and schemes

Scheme family	Examples	Common role	Post-quantum posture and cautions
EdDSA	Ed25519, Ed448	General-purpose signatures where ecosystem support exists	Quantum-vulnerable; deterministic signing helps avoid random nonce failures, but context binding still matters.
ECDSA	ECDSA P-256, ECDSA secp256k1	TLS, certificates, blockchain transactions	Quantum-vulnerable; nonce reuse or biased nonces can expose the private key.
Schnorr-style signatures	BIP-340 Schnorr, protocol-specific Schnorr variants	Blockchains, multisignatures, zero-knowledge protocols	Quantum-vulnerable; batch verification and multisignature variants need careful domain separation.
RSA-PSS	RSA Probabilistic Signature Scheme	Legacy public-key infrastructure and compatibility	Quantum-vulnerable; prefer PSS over older RSA PKCS #1 v1.5 signatures in new RSA designs.

Scheme family	Examples	Common role	Post-quantum posture and cautions
BLS signatures	BLS12-381 or BN254 deployments	Aggregatable signatures and threshold signing	Quantum-vulnerable and pairing-based; subgroup checks and domain separation are critical.
ML-DSA	Module-lattice signature standard	Post-quantum migration	Plausibly post-quantum; larger keys and signatures affect protocol design.
SLH-DSA	Stateless hash-based signature standard	Conservative post-quantum signatures	Plausibly post-quantum; signatures are large and performance differs sharply from elliptic-curve schemes.
Falcon / FN-DSA	Compact lattice signature selected for ongoing NIST standardization	Future post-quantum option where smaller signatures matter	Not one of the three finalized 2024 FIPS standards; track FIPS 206 status before treating as finalized.
Legacy signatures	DSA, RSA PKCS #1 v1.5 signatures	Compatibility and verification of old artifacts	Keep as legacy context; do not present as a modern default.

Assumptions

The signature scheme must resist forgery, the private key must remain secret, and verifiers must bind the public key to the right signer, protocol, message format, and domain.

Post-quantum posture

Depends on the signature scheme. RSA, ECDSA, EdDSA, and Schnorr-style signatures are quantum-vulnerable, while finalized post-quantum signature standards such as ML-DSA and SLH-DSA are designed for post-quantum migration. Falcon/FN-DSA is selected for ongoing standardization, so its deployment status should be checked separately.

Confidence model

Confidence comes from the signer controlling the private key, verifiers binding the public key to the right identity and context, and the signature scheme resisting forgery.

Failure modes and anti-patterns

- Signing ambiguous encodings.
- Reusing nonces in schemes where nonce uniqueness is required.
- Failing to bind signatures to domain, chain, or protocol context.

Further reading

- Boneh and Shoup, "A Graduate Course in Applied Cryptography."
- NIST FIPS 204, "Module-Lattice-Based Digital Signature Standard."
- NIST FIPS 205, "Stateless Hash-Based Digital Signature Standard."
- NIST CSRC, "Post-Quantum Cryptography Project."

Public-Key Encryption

One-sentence intuition

Public-key encryption lets anyone encrypt to a public key while only the private key holder can decrypt.

Security properties

- Confidentiality of plaintexts under the chosen security model.
- Sometimes sender anonymity or ciphertext unlinkability, depending on the construction and context.

What it does not provide

- Sender authentication.
- Message validity beyond successful decryption.
- Protection against metadata leaks.
- Safety if keys are misbound to identities.

Use cases

- Secure messaging.
- Hybrid encryption.
- Encrypted ballots.
- Key exchange support, depending on the protocol.

Concrete algorithms and schemes

Scheme or suite	Typical role	Key differences and cautions
RSA-OAEP	Legacy public-key encryption and hybrid encryption	Quantum-vulnerable; safe padding is mandatory and raw RSA is not encryption.
HPKE	Standard hybrid public-key encryption framework	Composes KEM, KDF, and AEAD choices; security depends on authenticated public keys and mode selection.
ECIES-style schemes	Hybrid encryption over elliptic-curve key agreement	Quantum-vulnerable and variant-heavy; interoperability and authentication details vary.
Integrated encryption in protocols	TLS 1.3, Signal-style sessions, Noise patterns	Public-key operations establish keys; application data is protected with symmetric encryption.
Post-quantum KEM-based hybrids	ML-KEM plus AEAD through a key schedule	Plausibly post-quantum at the KEM layer; ciphertext size, failure behavior, and authentication still matter.
Legacy RSA encryption	RSAES-PKCS1-v1_5	Compatibility only; historically fragile against padding-oracle mistakes.

Assumptions

The scheme must meet the intended security notion, public keys must be authenticated, private keys must remain secret, and encryption must be used through a safe scheme or hybrid construction rather than raw textbook operations.

Post-quantum posture

Depends on the scheme. RSA and elliptic-curve public-key encryption or key agreement are quantum-vulnerable. Post-quantum key encapsulation mechanisms such as ML-KEM are designed for migration, but protocol integration still matters.

Confidence model

Confidence comes from key authenticity, correct encryption or encapsulation, secure private-key handling, and the recipient's ability to decrypt. If the public key is bound to the wrong party, confidentiality can fail even when the encryption algorithm is sound.

Failure modes and anti-patterns

- Using raw textbook encryption instead of authenticated, padded, or hybrid constructions.
- Forgetting key authentication.
- Treating encrypted data as valid without additional checks or proofs.

Further reading

- Boneh and Shoup, "A Graduate Course in Applied Cryptography."
- NIST FIPS 203, "Module-Lattice-Based Key-Encapsulation Mechanism Standard."

Key Encapsulation and Exchange

One-sentence intuition

Key encapsulation and key exchange let parties establish shared secret material over an insecure channel.

Security properties

- Shared secret establishment.
- Forward secrecy in protocols designed for ephemeral secrets.
- Authentication when combined with signatures, certificates, pre-shared keys, or authenticated transcripts.

What it does not provide

- Entity authentication by itself.
- Application-data encryption without a subsequent key schedule and encryption layer.
- Metadata privacy for who talked to whom and when.

Assumptions

The scheme-specific hardness assumption must hold, public keys or identities must be authenticated when authentication is required, and derived keys must be bound to the full transcript.

Post-quantum posture

Depends on the concrete mechanism. Diffie-Hellman and elliptic-curve Diffie-Hellman are quantum-vulnerable, while standardized post-quantum KEMs such as ML-KEM are designed for migration.

Confidence model

Confidence comes from the key-establishment assumption, authentication binding, fresh ephemeral secret handling, and correct derivation of application keys from the shared secret.

Use cases

- Hybrid public-key encryption.
- Transport security handshakes.
- Secure messaging session setup.

Concrete algorithms and schemes

Mechanism	Family	Typical role	Key differences and cautions
X25519	Elliptic-curve Diffie-Hellman	Modern key agreement in protocols and libraries	Quantum-vulnerable; usually simple and robust when used through established libraries.
X448	Elliptic-curve Diffie-Hellman	Higher-security-margin key agreement	Quantum-vulnerable; less widely deployed than X25519.
P-256 ECDH	Elliptic-curve Diffie-Hellman	TLS and standards-oriented environments	Quantum-vulnerable; point validation and library correctness matter.
FFDHE	Finite-field Diffie-Hellman groups	Compatibility and standards profiles	Quantum-vulnerable; use reviewed safe-prime groups, not ad hoc parameters.
ML-KEM	Module-lattice KEM	Post-quantum key encapsulation	Plausibly post-quantum; protocol designers must handle larger keys and ciphertexts.
HQC	Code-based KEM selected for ongoing NIST standardization	Backup or alternative post-quantum KEM family	Selected by NIST in 2025 for future standardization; not a finalized FIPS standard as of this review.
HPKE KEM suites	KEM plus KDF plus AEAD framework	Hybrid encryption and application protocols	HPKE is a composition framework; the selected KEM determines posture.
Hybrid classical/PQ exchange	Classical ECDH plus ML-KEM or similar	Migration period key establishment	Reduces single-assumption risk, but transcript binding and failure handling must be explicit.

Failure modes and anti-patterns

- Establishing a key with an unauthenticated attacker.
- Failing to bind the transcript, identities, and algorithm choices into derived keys.
- Reusing ephemeral secrets where freshness is required.

Further reading

- Boneh and Shoup, "A Graduate Course in Applied Cryptography."
- NIST FIPS 203, "Module-Lattice-Based Key-Encapsulation Mechanism Standard."
- NIST, "NIST Selects HQC as Fifth Algorithm for Post-Quantum Encryption."

Secret Sharing

One-sentence intuition

Secret sharing splits a secret into shares so that only an authorized subset can reconstruct it.

In a threshold scheme, a secret can be split so that any `t` shares reconstruct it, while fewer than `t` shares should reveal nothing useful:

$$\text{threshold} = t \quad \text{out of} \quad n$$

For Shamir-style secret sharing, the dealer can choose a random polynomial whose constant term is the secret:

$$f(z) = s + a_1 z + \cdots + a_{t-1} z^{t-1}$$

Each participant receives one point on that polynomial:

$$\text{share}_i = (i, f(i))$$

Security properties

- Confidentiality against parties below the reconstruction threshold.
- Availability when enough shares survive.

Assumptions

Shares must be generated with correct randomness, distributed over authenticated channels, stored independently, and reconstructed only under the intended threshold and governance rules.

Post-quantum posture

Plausible for the information-theoretic core of schemes such as Shamir secret sharing. The full system may still depend on quantum-vulnerable authentication, transport encryption, signatures, or storage controls.

Confidence model

Confidence is threshold-based. Fewer than `t` shares should not reveal the secret; at least `t` valid shares can reconstruct it. Availability fails if too many shares are lost, and confidentiality fails if enough shareholders collude.

What it does not provide

- Authentication of shares unless added.
- Protection against maliciously corrupted shares unless verification is added.

- Automatic key rotation or operational security.

Use cases

- Threshold key custody.
- Backup and recovery.
- Distributed decryption.
- MPC building blocks.

Concrete algorithms and schemes

Scheme	Typical role	Key differences and cautions
Shamir secret sharing	Threshold sharing over a finite field	Information-theoretic privacy below threshold; requires authenticated distribution and careful reconstruction.
Additive secret sharing	MPC and simple split-control workflows	Simple and efficient, but usually needs all shares or protocol-specific reconstruction.
Feldman verifiable secret sharing	Publicly checkable share consistency	Adds public verification, but commitments can reveal structure and rely on discrete-logarithm assumptions.
Pedersen verifiable secret sharing	Verifiable sharing with hiding commitments	Hides coefficients better than Feldman-style commitments, but inherits generator and group assumptions.
Distributed key generation	Threshold key creation without one dealer knowing the whole secret	Protocol, not just a sharing algorithm; participant authentication and abort handling dominate risk.

Failure modes and anti-patterns

- Losing too many shares.
- Letting one organization control enough shares.
- No process for detecting invalid shares.

Further reading

- Adi Shamir, "How to Share a Secret."
- Boneh and Shoup, "A Graduate Course in Applied Cryptography."

Structured Primitives

Structured Primitives Overview

Structured primitives add algebraic, access-control, timing, or delegation structure to basic cryptographic building blocks.

Examples

- Homomorphic commitments preserve arithmetic relationships.
- Homomorphic encryption computes on encrypted values.
- Threshold cryptography distributes authority across parties.
- Functional encryption reveals only an approved function of encrypted data.
- Verifiable delay functions make computation take sequential time.

Safety note

The extra structure is useful because it exposes controlled relationships. The same structure can create surprising failure modes if the protocol forgets to constrain inputs or metadata.

Homomorphic Commitments

One-sentence intuition

A homomorphic commitment lets commitments be combined in ways that correspond to operations on the hidden values.

Use cases

- Confidential sums.
- Private tallying.
- Range proof systems.

What it does not provide

- Proof that hidden values are in a valid range.
- Authentication.
- Protection against maliciously crafted commitments without additional checks.

Assumptions

The commitment scheme must retain its hiding and binding properties under the allowed homomorphic operation, and the surrounding protocol must define the arithmetic domain being committed to.

Post-quantum posture

Depends on the construction. Pedersen-style homomorphic commitments are quantum-vulnerable because they rely on discrete logarithms. Other commitment families need separate classification.

Confidence model

Confidence comes from the commitment binding and hiding assumptions plus explicit constraints on the hidden arithmetic domain. Homomorphic structure is useful only when invalid values are ruled out elsewhere.

Concrete schemes and families

Scheme	Homomorphic shape	Typical role	Key differences and cautions
Pedersen commitments	Additive over committed values	Confidential amounts, private tallying, range proofs	Mature and efficient, but quantum-vulnerable and generator setup-sensitive.
Pedersen vector commitments	Linear relations over committed vectors	Inner-product proofs and multi-value commitments	Requires independent generators or a sound generator-derivation process.
KZG commitments	Polynomial openings and linear combinations	Rollups, data availability, succinct polynomial proofs	Compact but pairing-based and usually setup-dependent.

Scheme	Homomorphic shape	Typical role	Key differences and cautions
Inner-product-argument commitments	Vector and polynomial commitments	Bulletproof-style systems and transparent-ish vector commitments	Avoids pairing setup in common forms, but proof sizes and verification costs differ.
Merkle commitments	Set/list commitment via hashes	Membership proofs and sparse state commitments	Not algebraically homomorphic, but often used as the hash-based alternative when homomorphism is not required.

Failure modes and anti-patterns

Homomorphism can let invalid values cancel or wrap unless the protocol adds range proofs and clear arithmetic domains.

Further reading

- Torben Pryds Pedersen, "Non-Interactive and Information-Theoretic Secure Verifiable Secret Sharing."
- Pedersen commitments

Homomorphic Encryption

One-sentence intuition

Homomorphic encryption allows computation on ciphertexts so that decrypting the result reveals the result of a computation on the plaintexts.

At a high level, evaluation over ciphertexts should agree with evaluating the function over plaintexts:

$$\mathrm{Dec}_{sk}(\mathrm{Eval}(f, c_1, \ldots, c_n)) = f(m_1, \ldots, m_n)$$

where each ciphertext hides a message:

$$c_i = \mathrm{Enc}_{pk}(m_i)$$

Types

- Partially homomorphic encryption supports limited operations.
- Somewhat homomorphic encryption supports bounded computations.
- Fully homomorphic encryption (FHE) supports general computation in principle.

For an additive homomorphic scheme, the useful shape is:

$$\mathrm{Dec}_{sk}(c_1 \oplus c_2) = m_1 + m_2$$

Post-quantum posture

Plausible for many lattice-based homomorphic encryption families, assuming appropriate parameters and implementations. The posture still depends on the concrete scheme, security level, and whether surrounding signatures, proofs, or key-management layers are post-quantum.

Confidence model

Confidence usually comes from a scheme-specific hardness assumption, correct parameter selection, secure key generation, and protection of decryption keys. In threshold or multi-key settings, the confidence model also includes the trustee or participant threshold.

What it does not provide

- Automatic input validity.
- Protection against malicious outputs without verification.
- Metadata privacy.
- Practical efficiency for every workload.

Assumptions

The scheme-specific hardness assumption, parameter set, noise budget, key-management model, and implementation side-channel posture must all match the workload and threat model.

Use cases

- Private tallying.
- Confidential analytics.
- Encrypted computation services.

Concrete schemes and families

Family	Computation style	Typical role	Key differences and cautions
BFV	Exact arithmetic over integers modulo a plaintext modulus	Private tallying and exact arithmetic workloads	Good for exact values; parameter selection and batching shape performance.
BGV	Exact arithmetic with leveled homomorphic evaluation	Exact encrypted computation	Similar use space to BFV, with different noise-management techniques.
CKKS	Approximate arithmetic over real or complex values	Machine learning and statistical analytics	Approximate results are part of the design; precision and scale management are security-relevant engineering choices.
TFHE / FHEW-style schemes	Boolean or small-gate bootstrapped computation	Bit-level computation and programmable bootstrapping	Can support frequent bootstrapping; performance profile differs from arithmetic-circuit schemes.
Threshold HE variants	Shared decryption key across trustees	Private tallying and multi-party analytics	Adds a `t-of-n` confidence model on top of the encryption scheme.

Failure modes and anti-patterns

- Choosing parameters that do not meet the security or correctness target.
- Ignoring noise growth or implementation limits.
- Revealing too much through outputs or repeated queries.

Further reading

- Craig Gentry, "Fully Homomorphic Encryption Using Ideal Lattices."
- Boneh and Shoup, "A Graduate Course in Applied Cryptography."

Threshold Cryptography

One-sentence intuition

Threshold cryptography distributes a cryptographic power across several parties so that a quorum is required to act.

What it can provide

- Reduced single-key compromise risk.
- Distributed signing or decryption.
- Better availability if some parties fail.

What it does not provide

- Trustlessness.
- Protection if the threshold colludes.
- Simple operations or recovery.
- Metadata privacy by itself.

Assumptions

The protocol must generate and protect shares correctly, define the threshold and recovery process, authenticate participants, and handle share refresh, replacement, and audit procedures.

Post-quantum posture

Depends on the underlying primitive. Threshold ECDSA or threshold Schnorr is quantum-vulnerable. Threshold versions of post-quantum signatures, encryption, or key encapsulation need separate analysis and are not automatically available just because a single-party primitive exists.

Confidence model

Confidence is `t-of-n`: the system assumes fewer than `t` parties collude for privacy or key misuse resistance, and at least `t` parties are available for liveness. Distributed key generation, share custody, and recovery policy are part of the model.

Concrete schemes and protocols

Scheme or protocol family	Underlying primitive	Typical role	Key differences and cautions
Threshold BLS	Pairing-based signatures	Aggregated committee signatures and validator groups	Compact aggregation, but quantum-vulnerable and subgroup/domain rules matter.

Scheme or protocol family	Underlying primitive	Typical role	Key differences and cautions
FROST	Schnorr-style signatures	Efficient threshold signing	Quantum-vulnerable; participant binding, nonce handling, and signing rounds are critical.
Threshold ECDSA	ECDSA	Custody and blockchain signing where ECDSA is fixed by the ecosystem	More complex than threshold Schnorr; protocol implementation risk is high.
Distributed key generation	Secret sharing plus verification	Creating a threshold key without one dealer	Setup protocol must handle malicious participants and aborts.
Threshold decryption	Public-key encryption or homomorphic encryption	Voting, private tallying, escrowed decryption	Privacy and liveness depend on trustee threshold and share verification.
Threshold post-quantum schemes	Scheme-specific research and engineering	Migration target	Not automatic; each post-quantum primitive needs its own threshold design and maturity assessment.

Failure modes and anti-patterns

- Bad distributed key generation.
- Poor share custody.
- Unclear quorum governance.
- No plan for rotation, slashing, replacement, or disaster recovery.

Further reading

- Adi Shamir, "How to Share a Secret."
- Boneh and Shoup, "A Graduate Course in Applied Cryptography."

Functional Encryption

One-sentence intuition

Functional encryption lets a key reveal only a specific function of encrypted data, rather than the full plaintext.

Problem it solves

It aims to make decryption rights more precise. A party might learn an aggregate, classification, or score without learning each input.

What it does not provide

- General practicality for all functions.
- Simple deployment.
- Protection against outputs that are themselves revealing.
- A substitute for access-control design.

Assumptions

Assumptions are scheme-specific and often include a setup or key authority that issues function keys correctly. The system also assumes that the allowed function and repeated query pattern do not leak more than intended.

Maturity and deployment

Functional encryption remains largely research-stage for many general forms. Specialized forms may be more practical.

Post-quantum posture

Depends on the concrete construction. Functional encryption is a broad research area; posture should be classified per scheme and parameter set, not for the category as a whole.

Confidence model

Confidence often depends on a key authority or setup process that issues function keys. Even if the cryptography works, the allowed function can leak sensitive information, and repeated function outputs can become an inference channel.

Concrete schemes and subfamilies

Family	What the function key reveals	Typical role	Key differences and cautions
Identity-based encryption	Messages for a named identity	Key-management systems with a private-key generator	The authority can derive user keys, so issuer trust is central.
Attribute-based encryption	Decryption under an access policy or attribute set	Fine-grained encrypted access control	Policy privacy, revocation, and key abuse are system problems.
Inner-product functional encryption	Inner product or linear score	Private analytics and research prototypes	More specialized and practical than general FE, but still leakage-sensitive.
Predicate encryption	Whether encrypted attributes satisfy a predicate	Search and policy checks	The revealed predicate result can still leak sensitive information.
General functional encryption	Arbitrary functions in principle	Research-stage access to computed outputs	Mostly theoretical or highly specialized; do not present as deployable general access control.

Failure modes and anti-patterns

- Issuing function keys that reveal too much.
- Ignoring leakage from repeated function outputs.
- Treating research-stage general functional encryption as a deployable access-control layer.

Further reading

- Boneh, Sahai, and Waters, "Functional Encryption: Definitions and Challenges."

Timelock and VDFs

One-sentence intuition

Timelock tools and verifiable delay functions (VDFs) make information or outputs depend on the passage of sequential computation time.

A VDF can be read as a function that takes an input `x`, requires about `T` sequential steps to compute, and returns a result plus a proof:

$$(y, \pi) \leftarrow \mathrm{Eval}(x, T)$$

Verification should be much faster than evaluation:

$$\mathrm{Verify}(x, y, \pi) = 1$$

What VDFs provide

- A result that is slow to compute.
- A proof that is fast to verify, depending on the construction.

Post-quantum posture

Depends on the construction. Some VDF and timelock designs rely on assumptions whose post-quantum status is not the same as hash-based or lattice-based primitives. Treat each construction separately.

Confidence model

Confidence comes from the sequential-delay assumption, the verification equation, parameter selection, and any setup process. The model is not "trusted time"; it is an assumption about how much sequential work an adversary can complete.

What it does not provide

- Wall-clock fairness in every network setting.
- Protection against specialized hardware unless modeled.
- Confidentiality by themselves.

Assumptions

The delay parameter must reflect realistic sequential computation, the setup model must be sound for the chosen construction, and the protocol must account for network delay, denial of service, and hardware advantage.

Use cases

- Randomness beacons.
- Delayed reveal.
- Leader election protocols.

Concrete schemes and families

Scheme or family	Assumption shape	Typical role	Key differences and cautions
Repeated-squaring timelocks	Sequential squaring in an unknown-order group	Delayed disclosure and timelock puzzles	Delay depends on sequential work; setup and modulus/class-group choice matter.
Wesolowski VDF	Unknown-order group with compact proof	Publicly verifiable delay outputs	Compact verification, but assumption and setup choices must be explicit.
Pietrzak VDF	Unknown-order group with interactive/recursive proof structure	Verifiable delay outputs	Different proof and verification trade-offs from Wesolowski-style VDFs.
Class-group VDFs	Unknown-order class groups	Avoiding trusted RSA modulus generation	Parameter generation differs from RSA groups and still needs expert review.
Trusted delay services	External time authority or server	Engineering substitute for cryptographic delay	Not a VDF; confidence shifts to service trust and availability.

Failure modes and anti-patterns

- Underestimating hardware advantage.
- Confusing sequential work with trusted time.
- Ignoring denial-of-service and availability.

Further reading

- Boneh et al., "Verifiable Delay Functions."
- Wesolowski, "Efficient Verifiable Delay Functions."

Accumulators and Merkle Trees

One-sentence intuition

Accumulators and Merkle trees commit to a collection while supporting compact membership, and sometimes non-membership, proofs.

Use cases

- Certificate transparency.
- Blockchains and authenticated data structures.
- Anonymous membership sets.
- Airdrop eligibility lists.

What it does not provide

- Privacy of the set unless the design hides it.
- Freshness unless updates are authenticated.
- Protection against metadata leaks.

Assumptions

The set encoding must be canonical, roots must be authenticated, update rules must be clear, and verifiers must know which root or accumulator state is current.

Post-quantum posture

Depends on the accumulator. Merkle trees built from appropriate hash functions are plausibly post-quantum. RSA accumulators and elliptic-curve accumulators are quantum-vulnerable.

Confidence model

Confidence comes from the authenticated set root, update rules, and membership proof verification. Dynamic accumulators also need a freshness model so verifiers know which root is current.

Concrete schemes and data structures

Scheme or structure	Proof type	Typical role	Key differences and cautions
Binary Merkle tree	Inclusion proofs	Blockchains, transparency logs, allowlists	Hash-based and plausibly post-quantum; encoding and leaf/internal-node domain separation matter.
Sparse Merkle tree	Inclusion and non-inclusion over a large key space	Account/state commitments and nullifier sets	Proofs can be predictable in size; default empty nodes and key hashing must be specified.

Scheme or structure	Proof type	Typical role	Key differences and cautions
Merkle Mountain Range	Append-only inclusion proofs	Logs and append-only ledgers	Good for append-only histories; not the same update model as a mutable tree.
RSA accumulator	Compact membership witnesses	Credential revocation and set membership	Quantum-vulnerable; modulus generation and witness update rules are central.
Bilinear accumulator	Pairing-based membership witnesses	Specialized anonymous credential or proof systems	Quantum-vulnerable and pairing-based; setup and subgroup checks matter.
KZG/Verkle-style commitments	Vector openings	Compact authenticated state	Pairing-based and setup-sensitive in common forms.

Failure modes and anti-patterns

- Ambiguous tree encoding.
- No domain separation between leaves and internal nodes.
- Treating membership as authorization without checking context.

Further reading

- Ralph Merkle, "A Digital Signature Based on a Conventional Encryption Function."
- Boneh and Shoup, "A Graduate Course in Applied Cryptography."

Proof Systems

Proof Systems Overview

Proof systems let a prover convince a verifier that a statement is true. Some proof systems also hide the witness.

Examples

- Zero-knowledge proofs hide witnesses.
- Range proofs show a hidden value is within bounds.
- Membership proofs show inclusion in a set.
- SNARKs, STARKs, and Bulletproofs are proof-system families with different trade-offs.

Reading rule

Always identify the statement, the witness, public inputs, setup assumptions, verifier cost, prover cost, and metadata leaks.

Zero-Knowledge Proofs

One-sentence intuition

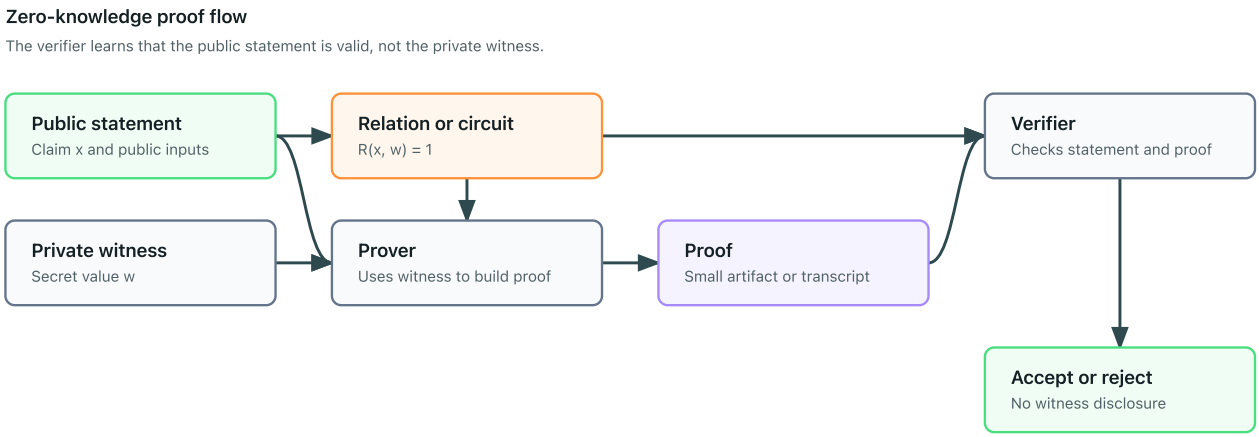
A zero-knowledge proof lets one party prove that a statement is true without revealing the secret information that makes it true.

Where it sits in the taxonomy

- Level: proof system
- Parent category: proof systems
- Related concepts: SNARKs, STARKs, Bulletproofs, range proofs, membership proofs

Problem it solves

Zero-knowledge proofs (ZKPs) are useful when a verifier needs confidence in a claim but should not learn the private witness behind the claim.



Statement vs witness

The statement is the public claim being proven. The witness is the private information that makes the statement true.

Many proof systems can be read as proving that there exists a private witness `w` such that a public relation accepts the public statement `x`:

$$\exists w : R(x, w) = 1$$

The verifier should learn that this relation is satisfied, not the witness itself.

Examples:

Statement	Witness
This commitment contains a number between 0 and 100	The committed number and randomness
I know a credential signed by an issuer	The credential and secret key
This encrypted vote is one of the allowed choices	The plaintext vote and encryption randomness

Core properties

- Completeness: honest proofs for true statements verify.
- Soundness: false statements should not verify except with negligible probability.
- Zero-knowledge: the proof should not reveal the witness beyond the truth of the statement.

What ZKPs do

- Prove membership in a set without revealing which member, depending on the construction.
- Prove that a hidden value is in a valid range.
- Prove that a vote, transaction, or credential use follows specified rules.
- Reduce trust in validators who would otherwise need to inspect private data.

What ZKPs do not provide

- Encryption of arbitrary data.
- Network anonymity.
- Protection against metadata leaks.
- A complete protocol by themselves.
- Safety if the statement being proven is the wrong statement.
- Protection against a compromised witness.

Family vs concrete proof systems

"Zero-knowledge proof" is a broad family. SNARKs, STARKs, and Bulletproofs are concrete proof-system families with different trade-offs in proof size, verifier cost, prover cost, setup assumptions, post-quantum posture, and implementation maturity.

Concrete proof-system families

Family or scheme	Setup model	Typical role	Key differences and cautions
Sigma protocols	Often interactive or Fiat-Shamir transformed	Knowledge proofs and identification-style protocols	Simple building blocks; transcript binding controls non-interactive security.
Groth16	Circuit-specific trusted setup	Very small proofs and fast verification	Pairing-based and quantum-vulnerable; setup is tied to the circuit.
PLONK-style systems	Often universal/updatable setup	General-purpose SNARK proving stacks	More flexible setup than Groth16-style systems, but assumptions and arithmetization choices vary.

Family or scheme	Setup model	Typical role	Key differences and cautions
STARKs	Transparent setup	Scalable transparent proofs	Usually hash-based and plausibly post-quantum; proofs are larger than many SNARKs.
Bulletproofs	No trusted setup in common forms	Range proofs and inner-product statements	Discrete-logarithm based and quantum-vulnerable; verification cost grows with statement size.
Folding and accumulation systems	Varies by construction	Recursive proofs and incremental verifiable computation	Maturity and assumptions are construction-specific; do not treat all folding schemes as interchangeable.

Minimal examples

- Membership: prove a secret appears in a committed list without revealing which entry.
- Valid vote: prove an encrypted ballot encodes one allowed choice.
- Range proof: prove a hidden amount is non-negative and below a limit.

Assumptions

Assumptions vary by proof system. Some systems require trusted setup, some rely on hash functions, some rely on elliptic curve assumptions, and some are designed around transparent setup.

Post-quantum posture

Depends on the proof system. Hash-based transparent systems such as many STARK-style systems are commonly treated as plausibly post-quantum, while many pairing-based SNARKs and discrete-logarithm-based range proofs are quantum-vulnerable.

Confidence model

Confidence comes from three layers: the proof-system assumptions, the correctness of the statement being proven, and the setup model. A proof can verify correctly while still proving the wrong statement for the application.

Failure modes and anti-patterns

- Proving a weak statement that does not match the system's real security goal.
- Ignoring public inputs that leak identity or linkage.
- Reusing witnesses, nullifiers, or randomness in linkable ways.
- Treating a proof system as a complete privacy protocol.
- Deploying research-stage proving systems without expert review.

Maturity and deployment

ZKPs are a mature field, but concrete systems vary from widely deployed to research-stage. Maturity must be evaluated per construction and implementation.

Related concepts

- Range proofs
- Membership proofs
- Nullifiers

Further reading

- Goldwasser, Micali, and Rackoff, "The Knowledge Complexity of Interactive Proof Systems."
- Ben-Sasson et al., "Scalable, transparent, and post-quantum secure computational integrity."
- Bünz et al., "Bulletproofs: Short Proofs for Confidential Transactions and More."

SNARKs, STARKs, and Bulletproofs

One-sentence intuition

SNARKs, STARKs, and Bulletproofs are families of proof systems that package zero-knowledge or succinct verification with different costs and assumptions.

Comparison snapshot

Family	Common strengths	Common trade-offs
SNARKs	Small proofs and fast verification	Some constructions need trusted setup or pairing assumptions
STARKs	Transparent setup and hash-based assumptions	Larger proofs and heavier verification than many SNARKs
Bulletproofs	No trusted setup and useful range proofs	Verification can be heavier for large statements

Concrete systems and families

System or family	Category	Setup model	Common use	Main caution
Groth16	Pairing-based SNARK	Circuit-specific trusted setup	Very small proofs in deployed ZK systems	Setup and circuit specificity dominate confidence.
PLONK-style systems	Polynomial-commitment SNARK	Often universal or updatable setup	General-purpose circuits and rollups	Exact assumptions depend on the commitment scheme and transcript design.
Marlin / Sonic-style systems	Universal-setup SNARK family	Universal structured setup	General circuits with reusable setup	Setup is reusable but still a setup assumption.
STARKs	Transparent proof system	Transparent	Scalable computation proofs	Larger proofs; hash and FRI parameters are security-critical.
FRI / DEEP-FRI	Low-degree testing component	Transparent	STARK-style proof systems	It is a component, not the full application statement.
Bulletproofs	Inner-product proof system	No trusted setup in common forms	Range proofs and confidential transactions	Discrete-logarithm based and quantum-vulnerable.
Halo / Nova-style systems	Accumulation or folding families	Varies	Recursive and incremental proofs	Rapidly evolving; maturity is implementation-specific.

Post-quantum posture

Depends on the family and construction. Many deployed pairing-based SNARKs are quantum-vulnerable. STARK-style systems are often treated as plausibly post-quantum when instantiated with appropriate hash functions. Bulletproof-style systems are usually discrete-logarithm based and therefore quantum-vulnerable.

Confidence model

Confidence comes from the proof-system assumptions, setup model, and statement design. Pairing-based SNARKs may require trusted or universal setup. STARK-style systems are usually transparent. Bulletproof-style systems avoid trusted setup but still rely on discrete-logarithm assumptions.

What it does not provide

- A correct statement automatically.
- Metadata privacy.
- Protection against implementation bugs.
- A complete protocol.

Assumptions

Assumptions are family-specific: pairing-based SNARKs often depend on elliptic-curve and setup assumptions, STARK-style systems typically depend on hash choices and transparent protocols, and Bulletproof-style systems usually depend on discrete-logarithm assumptions.

Failure modes and anti-patterns

- Choosing a proof family based only on proof size.
- Treating trusted setup, transparent setup, and universal setup as interchangeable.
- Proving a statement that omits important public inputs.
- Ignoring prover cost, verification cost, and implementation maturity.

Further reading

- Ben-Sasson et al., "Scalable, transparent, and post-quantum secure computational integrity."
- Bünz et al., "Bulletproofs: Short Proofs for Confidential Transactions and More."
- Goldwasser, Micali, and Rackoff, "The Knowledge Complexity of Interactive Proof Systems."

Range Proofs

One-sentence intuition

A range proof shows that a hidden value lies within an allowed interval.

Why it matters

Hidden values can be invalid. In private payments, a value may need to be non-negative. In voting, a ballot may need to be one of a small set of choices.

The typical statement is that a hidden value lies in a public interval:

$$m \in [0, 2^k - 1]$$

When the value is inside a commitment, the proof should be bound to that exact commitment:

$$C = \mathrm{Commit}(m; r)$$

Post-quantum posture

Depends on the proof system and commitment scheme. Bulletproof-style range proofs are usually discrete-logarithm based and quantum-vulnerable; hash-based or STARK-style approaches may be plausibly post-quantum if the full construction supports the required statement.

Confidence model

Confidence comes from public verification of the range statement, the soundness of the proof system, and correct binding to the commitment or ciphertext being constrained.

What it does not provide

- Authentication.
- Correct tallying by itself.
- Protection against metadata leaks.
- A guarantee that the range was the right policy choice.

Assumptions

The proof system must be sound, the proof must be bound to the exact commitment or ciphertext, and the range must be encoded without field wraparound or overflow ambiguity.

Use cases

- Confidential transactions.
- Private voting.

- Rate limits.
- Private statistics.

Concrete schemes and families

Scheme or family	Commitment/proof base	Typical role	Key differences and cautions
Bulletproof range proofs	Pedersen commitments plus inner-product arguments	Confidential transactions and hidden balances	No trusted setup in common forms, but discrete-logarithm based and quantum-vulnerable.
Boudot-style interval proofs	Integer commitments	Earlier range-proof constructions	Useful historically; parameter and efficiency trade-offs differ from Bulletproofs.
SNARK-based range proofs	Groth16, PLONK-style systems	Validity proofs inside larger circuits	Compact verification, but setup and statement correctness matter.
STARK-based range checks	STARK constraints and lookups	Transparent proof systems	Plausibly post-quantum when the full stack is; proof size and constraint design matter.
Lookup-based range checks	Plookup-style tables or custom lookup arguments	Modern circuit systems	Efficient for bounded values, but table binding and field encoding must be explicit.

Failure modes and anti-patterns

- Proving the wrong bound.
- Ignoring overflow or field wraparound.
- Failing to bind the proof to the correct commitment or context.

Further reading

- Bünz et al., "Bulletproofs: Short Proofs for Confidential Transactions and More."
- Boudot, "Efficient Proofs that a Committed Number Lies in an Interval."

Membership Proofs

One-sentence intuition

A membership proof shows that an item belongs to a committed set.

Use cases

- Proving eligibility.
- Showing inclusion in a Merkle tree.
- Anonymous membership when combined with zero knowledge.

What it does not provide

- Authorization policy by itself.
- Privacy unless the proof hides which member is used.
- Freshness unless the set commitment is current.

Assumptions

The set commitment must be authentic, leaf encoding must be unambiguous, and the verifier must know which root or accumulator state is current for the application.

Post-quantum posture

Depends on the construction. Hash-based Merkle inclusion proofs can be plausibly post-quantum with appropriate hashes, while many algebraic accumulators rely on assumptions that may be quantum-vulnerable.

Confidence model

Confidence comes from public verification of the set commitment, collision resistance or accumulator soundness, and correct binding between the proof and the policy context.

Concrete schemes and families

Scheme	Set commitment	Typical role	Key differences and cautions
Merkle inclusion proof	Merkle root	Allowlists, transparency logs, blockchain state	Hash-based and compact logarithmic proofs; reveals path information unless wrapped in zero knowledge.
Sparse Merkle proof	Sparse Merkle root	Large key spaces, nullifier sets, account state	Supports non-membership patterns; encoding and default nodes must be canonical.
RSA accumulator witness	RSA accumulator value	Compact set membership and revocation	Quantum-vulnerable and setup-sensitive; witness updates are operationally important.

Scheme	Set commitment	Typical role	Key differences and cautions
Bilinear accumulator witness	Pairing-based accumulator	Anonymous credentials and specialized protocols	Quantum-vulnerable; setup and subgroup checks matter.
KZG opening proof	Polynomial/vector commitment	Verkle-style state and data availability	Very compact openings; pairing and setup assumptions are central.
ZK membership proof	Merkle or accumulator proof inside a ZKP	Anonymous membership	Hides which member is used, but inherits proof-system and set-root assumptions.

Failure modes

- Using stale set roots.
- Ambiguous leaf encoding.
- Revealing the member through the proof path or metadata.

Further reading

- Ralph Merkle, "A Digital Signature Based on a Conventional Encryption Function."
- Boneh and Shoup, "A Graduate Course in Applied Cryptography."

Protocols

Protocols Overview

Protocols specify how parties interact. They combine primitives, messages, state transitions, checks, and failure handling.

Protocol questions

- Who participates?
- What are their inputs and outputs?
- What can each adversary observe or corrupt?
- What metadata is public?
- What happens if a party aborts?
- What assumptions are required?

Covered protocol families

- Anonymous credentials
- Nullifiers
- Oblivious pseudorandom functions
- Private set intersection
- Secure aggregation
- Multi-party computation
- Mixnets
- Electronic voting

Safety note

A protocol can fail even if every primitive inside it is sound.

Protocol coverage is intentionally not exhaustive. Missing or shallow protocol families are tracked in Editorial Maturity and Coverage.

Anonymous Credentials

Goal

Anonymous credentials let a user prove authorization or attributes without revealing a stable identity.

Participants

- Issuer.
- Holder.
- Verifier.

Inputs and outputs

The issuer gives a credential to a holder. Later, the holder proves selected claims to a verifier.

Building blocks

- Digital signatures or credential schemes.
- Zero-knowledge proofs.
- Revocation or status mechanisms.

Security goals

- Selective disclosure.
- Unlinkability between presentations, depending on the scheme.
- Issuer authenticity.

Non-goals

- Network anonymity.
- Coercion resistance.
- Protection if the holder exposes secrets.

Threat model

Designs must consider issuer-verifier collusion, verifier tracking, credential sharing, and revocation leakage.

Protocol sketch

1. The issuer verifies eligibility and issues a credential.
2. The holder stores the credential and secret.
3. The holder proves a claim to a verifier.
4. The verifier checks the proof, issuer, context, and revocation status.

Trust assumptions

The issuer may be trusted to issue correctly. Some systems require non-collusion or privacy-preserving revocation infrastructure.

Post-quantum posture

Depends on the credential signature scheme, presentation proof, accumulator, and revocation mechanism. The anonymous-credential pattern itself is not enough to determine post-quantum posture.

Confidence model

Confidence usually depends on a trusted issuer, holder-controlled secrets, verifier-side checks, and revocation infrastructure. Some designs also require non-collusion between issuer and verifier to preserve privacy.

Metadata leaks

Timing, verifier identity, IP addresses, rare attributes, and revocation checks can identify the holder.

Failure modes

- Attributes are too identifying.
- Revocation checks create tracking.
- Presentations are not bound to the right context.
- Credentials are transferable when the system assumes they are not.

Variants

- Selective disclosure credentials.
- Anonymous credentials with nullifiers.
- Unlinkable presentations.

Concrete schemes and systems

Scheme or family	Typical role	Key differences and cautions
CL signatures / Idemix-style credentials	Anonymous credentials with selective disclosure	Mature academic lineage; issuer trust and revocation design are central.
BBS+ signatures	Selective-disclosure credentials and unlinkable presentations	Pairing-based and quantum-vulnerable; useful for compact multi-message disclosure.
SD-JWT / selective-disclosure verifiable credentials	Practical web credential ecosystems	Easier web integration, but not automatically unlinkable against issuer/verifier correlation.
ZK credential systems	Credentials proven inside a zero-knowledge proof	Can hide more metadata, but inherits proof-system assumptions and circuit correctness risk.
Accumulator-based revocation	Private or semi-private status checks	Revocation can reintroduce linkability if freshness checks are not designed carefully.

Where it is used

- Private access control.
- Eligibility proofs.
- Age or membership claims.

Further reading

- Camenisch and Lysyanskaya, "An Efficient System for Non-transferable Anonymous Credentials with Optional Anonymity Revocation."
- Goldwasser, Micali, and Rackoff, "The Knowledge Complexity of Interactive Proof Systems."

Nullifiers

Goal

A nullifier helps enforce one-person-one-action, or one-secret-one-action, without directly revealing the identity behind the action.

Participants

- User: holds a secret or credential.
- Verifier or system: checks whether the action is valid.
- Public registry or ledger: records used nullifiers to prevent reuse.

Inputs and outputs

- Input: a user secret and a context such as an election, airdrop, or application identifier.
- Output: a public nullifier value.

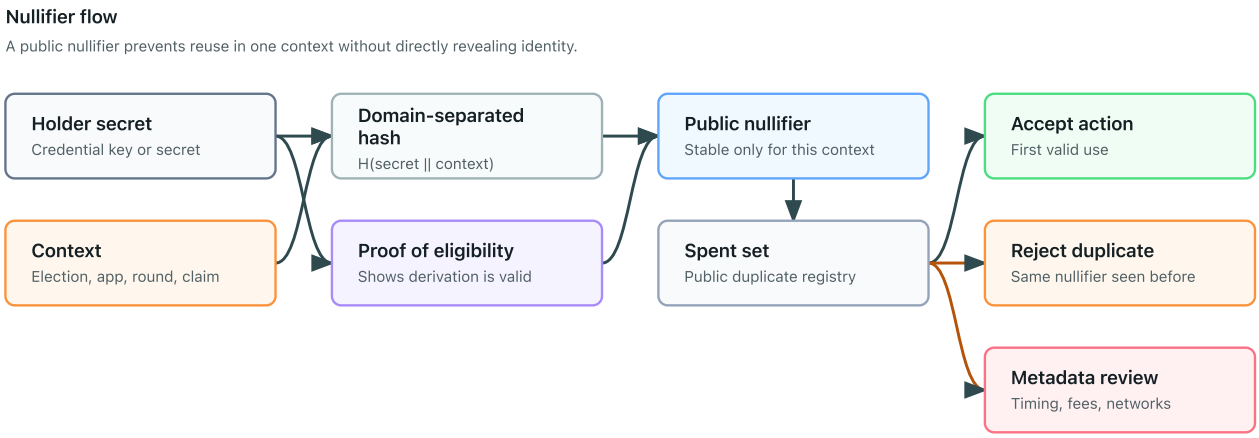
A common mental model is:

$$N = H(\text{secret} \,\|\, \text{context})$$

For the same secret, changing the context should change the nullifier:

$$H(s \,\|\, c_1) \neq H(s \,\|\, c_2)$$

Real systems normally derive nullifiers inside a larger protocol and may prove correctness with a zero-knowledge proof.



Building blocks

- Hash functions.

- Anonymous credentials or membership proofs.
- Zero-knowledge proofs.
- A public spent-nullifier set.

Security goals

- Prevent double use in the same context.
- Avoid revealing the user's identity directly.
- Avoid linking actions across different contexts when domain separation is correct.

Non-goals

- A nullifier does not by itself prove eligibility.
- A nullifier does not hide network metadata.
- A nullifier does not prevent coercion.
- A nullifier does not protect a user whose secret is compromised.

Threat model

The system assumes adversaries may observe public nullifiers and try to link them to users or reuse credentials. The design must prevent cross-context linkage and reject duplicate nullifiers.

Protocol sketch

1. A user obtains or holds an eligibility secret.
2. For a specific context, the user derives a nullifier.
3. The user proves, often in zero knowledge, that the nullifier was derived from an eligible secret.
4. The system checks that the nullifier has not appeared before.
5. The system records the nullifier after accepting the action.

Trust assumptions

Trust assumptions depend on the issuer, eligibility registry, proof system, and public registry. If the issuer can deanonymize credentials or the registry censors submissions, privacy and availability may fail.

Post-quantum posture

Depends on the surrounding stack. A hash-derived nullifier can be plausibly post-quantum if the hash function and parameters are appropriate, but eligibility proofs, credential signatures, and membership proofs may rely on quantum-vulnerable assumptions.

Confidence model

Confidence comes from the holder secret, domain-separated context, public spent-nullifier registry, and a proof that the nullifier was derived from an eligible secret. The nullifier alone only supports non-reuse; it does not establish eligibility.

Metadata leaks

Nullifiers can still be linked through timing, network address, account funding, transaction fees, wallet behavior, or reused contexts.

Failure modes

- Reusing a context across applications links actions.
- Omitting eligibility proofs allows arbitrary nullifiers.
- Weak secrets allow guessing attacks.
- Poor domain separation creates accidental cross-context linkage.
- Public ordering can expose behavioral patterns.

Variants

- Per-application nullifiers.
- Per-election nullifiers.
- Rate-limited nullifiers.
- Nullifiers derived from anonymous credentials.

Concrete constructions and patterns

Construction	Typical role	Key differences and cautions
Hash nullifier	One-use marker derived from a secret and context	Simple shape, but weak secrets or poor domain separation can make it guessable or linkable.
ZK-derived nullifier	Public nullifier plus proof of correct derivation	Common in anonymous membership systems; statement must bind eligibility, context, and nullifier.
Serial-number e-cash pattern	Preventing double-spend of anonymous tokens	Mature idea, but issuance, spend, and revocation details vary by system.
Semaphore-style nullifier	Anonymous signal or one-action-per-group pattern	Tied to Merkle membership and zero-knowledge proof assumptions.
Rate-limit nullifier	Allows limited actions per epoch or context	Context design controls linkability and replay boundaries.

Where it is used

- Private voting.
- Anonymous airdrops.
- Anonymous signaling.
- Private access systems with rate limits.

Further reading

- Camenisch and Lysyanskaya, "An Efficient System for Non-transferable Anonymous Credentials with Optional Anonymity Revocation."

- Goldwasser, Micali, and Rackoff, "The Knowledge Complexity of Interactive Proof Systems."

Oblivious Pseudorandom Functions

Goal

An oblivious pseudorandom function (OPRF) lets a client learn `F(k, x)` for its private input `x` and a server-held key `k`, without the server learning `x` or the output and without the client learning `k`.

Participants

- Client: holds the private input.
- Server: holds the PRF key.
- Optional verifier role: in verifiable variants, the client checks that the server used the expected key.

Inputs and outputs

- Client input: private value `x`.
- Server input: secret PRF key `k`.
- Output: the client learns `F(k, x)`.
- Public parameters: group, hash-to-group/hash-to-scalar rules, domain separation tags, and protocol mode.

Building blocks

- A pseudorandom function abstraction.
- A group or other algebraic structure suitable for the construction.
- Blinding and unblinding operations.
- Hash-to-group or encoding rules.
- Optional zero-knowledge proof of correct evaluation for verifiable OPRFs.

Security goals

- Client input privacy from the server.
- Server key privacy from the client.
- Pseudorandomness of the output to parties without the key.
- Verifiability in VOPRF variants, where the client can check the server used the advertised key.

Non-goals

- Hiding that a client contacted the server.
- Preventing online rate limits or denial of service.
- Protecting low-entropy inputs from guessing once outputs are exposed.
- Providing anonymity by itself.
- Making the server stateless or non-censoring.

Threat model

The usual model considers a malicious or curious server that should not learn the client's input and a malicious client that should not learn the server key or evaluate the PRF on arbitrary values without interaction. Application-level privacy also depends on transport privacy, rate limiting, replay rules, and whether inputs have enough entropy.

Protocol sketch

1. The client maps `x` into the protocol domain and blinds it.
2. The client sends the blinded value to the server.
3. The server evaluates using key `k` on the blinded value.
4. The server returns the evaluated value, and in verifiable modes also returns a proof.
5. The client verifies any proof, unblinds the response, and derives `F(k, x)`.

Trust assumptions

The algebraic assumptions for the chosen construction must hold, encodings and domain separation must be unambiguous, the server key must remain secret, and implementations must validate group elements and reject malformed inputs.

Post-quantum posture

Depends on the construction. Standard prime-order-group OPRFs inherit discrete-logarithm vulnerability to large quantum computers. Post-quantum OPRF designs exist as research and engineering work, but posture must be assessed per construction and parameter set.

Confidence model

Confidence comes from mathematical-assumption hardness, server key secrecy, public parameter correctness, and client-side verification in VOPRF or POPRF modes. Operational confidence also depends on server availability and abuse controls.

Metadata leaks

- Client-server contact timing.
- Server identity.
- Request volume and retry patterns.
- Public input in partially oblivious variants.
- Account, network, or payment metadata outside the OPRF transcript.

Failure modes

- Reusing an OPRF output as if it were an authentication token without binding it to context.
- Forgetting domain separation between applications.
- Accepting invalid group elements or non-canonical encodings.

- Using low-entropy inputs without a rate-limited or password-hardened design.
- Assuming OPRFs provide anonymity or hide network metadata.
- Treating a non-verifiable OPRF as if the server key were publicly committed.

Variants

- OPRF: client learns the PRF output; server learns neither input nor output.
- VOPRF: client can verify the server used the expected key.
- POPRF: public input is included in the PRF computation, useful for context binding.
- Application-specific designs: password-hardening services, privacy-preserving tokens, and private set intersection variants.

Where it is used

- Password-hardened authentication and secret recovery.
- Privacy-preserving rate limits or token issuance.
- Private set intersection.
- Credential and anonymous-token systems.
- Some privacy-preserving contact discovery designs.

Further reading

- RFC 9497, "Oblivious Pseudorandom Functions (OPRFs) Using Prime-Order Groups."
- Jarecki and Liu, "Efficient Oblivious Pseudorandom Function with Applications to Adaptive OT and Secure Computation of Set Intersection."
- Boneh and Shoup, "A Graduate Course in Applied Cryptography."

Private Set Intersection

Goal

Private set intersection (PSI) lets parties learn the overlap between their private sets while revealing as little as possible about non-overlapping elements.

Participants

- Two or more set holders.
- Optional helper or server roles in delegated, outsourced, or multi-party variants.
- Optional auditor or verifier in variants that prove correct computation.

Inputs and outputs

- Inputs: each participant's private set.
- Outputs: the intersection, the size of the intersection, or a function of matching records, depending on the protocol.
- Public parameters: hash functions, encodings, protocol suite, security level, and sometimes public keys or setup material.

Building blocks

- Oblivious pseudorandom functions (OPRFs).
- Diffie-Hellman-style blinding or public-key encryption.
- Oblivious transfer.
- Garbled circuits or multi-party computation.
- Homomorphic encryption in some constructions.
- Hashing, canonical encoding, and domain separation.

Security goals

- Hide non-matching elements under the stated threat model.
- Reveal only the intended output: intersection, cardinality, or approved function.
- Bind comparisons to canonical encodings so equal items match and distinct items do not accidentally collide.
- Support semi-honest or malicious security depending on the construction.

Non-goals

- Hiding the fact that a PSI run happened.
- Hiding set sizes unless the protocol pads or otherwise protects them.
- Preventing inference from the intersection itself.
- Handling fuzzy, approximate, or dirty data safely by default.

- Providing fairness if one party aborts after learning output.

Threat model

PSI protocols vary sharply. A semi-honest protocol assumes participants follow the protocol but try to learn extra information from transcripts. A malicious-secure protocol must handle malformed inputs, selective aborts, and attempts to bias or expand the result. Multi-party, delegated, and asymmetric PSI variants add different collusion and availability assumptions.

Protocol sketch

One common OPRF-style two-party shape is:

1. The client encodes and blinds each element in its set.
2. The server evaluates an OPRF on blinded elements without seeing them.
3. The client unblinds outputs and compares them with server-provided encodings of the server set.
4. The client learns matching elements or intersection size, depending on the design.
5. Malicious-secure variants add proofs, consistency checks, or cut-and-choose-style defenses.

Trust assumptions

Inputs must be encoded canonically, the chosen construction's assumptions must hold, each party must use the agreed security parameters, and the output policy must match the privacy claim. If helper servers are used, the protocol must state whether privacy depends on non-collusion.

Post-quantum posture

Depends on the construction. Diffie-Hellman and many OPRF-based PSI protocols are quantum-vulnerable. PSI from symmetric-key multi-party computation or post-quantum assumptions may be plausible, but concrete posture depends on the protocol, parameters, and authentication layer.

Confidence model

Confidence may come from mathematical assumptions, a semi-honest or malicious-secure proof, public verification of commitments or proofs, or non-collusion among helper services. These models are not interchangeable.

Metadata leaks

- Set sizes unless padded.
- Timing, retry, and abort behavior.
- Which party learns the output.
- Intersection size if cardinality is revealed.
- Network identifiers and operational logs.
- Distributional clues from rare or predictable elements.

Failure modes

- Treating PSI as "privacy preserving" without stating exactly what output is revealed.
- Comparing low-entropy identifiers that are easy to guess offline.
- Ignoring set-size leakage.
- Using inconsistent normalization, causing false matches or missed matches.
- Running repeated PSI queries that allow differencing attacks.
- Using a semi-honest protocol against malicious parties.
- Revealing too much through post-processing of matched records.

Variants

- PSI with intersection output.
- PSI-cardinality, where only the intersection size is revealed.
- PSI with associated payloads, where matching records unlock extra values.
- Multi-party PSI.
- Delegated or outsourced PSI.
- Fuzzy or approximate PSI, which usually has stronger leakage concerns.

Where it is used

- Private contact discovery.
- Fraud and abuse signal sharing.
- Privacy-preserving record linkage.
- Audience overlap and measurement.
- Federated learning cohort construction.
- Credential or allowlist checks when full lists should not be shared.

Further reading

- Freedman, Nissim, and Pinkas, "Efficient Private Matching and Set Intersection."
- RFC 9497, "Oblivious Pseudorandom Functions (OPRFs) Using Prime-Order Groups."
- Bonawitz et al., "Practical Secure Aggregation for Privacy-Preserving Machine Learning."
- Canetti, "Universally Composable Security; A New Paradigm for Cryptographic Protocols."

Secure Aggregation

Goal

Secure aggregation lets a server learn an aggregate of client values without learning each individual value.

Participants

- Clients that hold private values.
- Aggregator or server.
- Sometimes helper servers.

Inputs and outputs

Clients input values. The server learns an aggregate such as a sum or average.

Building blocks

- Secret sharing.
- Pairwise masks.
- Authentication.
- Dropout handling.

Security goals

- Hide individual values from the aggregator under the chosen collusion model.
- Recover the aggregate if enough clients complete the protocol.

Non-goals

- Privacy for tiny groups.
- Protection against malicious values unless validation is added.
- Differential privacy unless added separately.

Threat model

The model must specify how many clients or servers may collude, whether clients can drop out, and whether malicious clients can submit malformed updates.

Protocol sketch

1. Clients authenticate and agree on masks.
2. Each client sends a masked value.
3. The server combines masked values.

4. Masks cancel or are reconstructed according to the protocol.
5. The server learns only the aggregate.

Trust assumptions

Assumptions depend on the number of clients, dropout thresholds, authentication, and helper-server model.

Post-quantum posture

Depends on the transport, authentication, key agreement, and masking primitives. The aggregation pattern can be made from post-quantum components, but many deployed channels and signatures may not be post-quantum today.

Confidence model

Confidence comes from the collusion threshold, dropout model, authentication, and whether helper servers are assumed not to collude. Small cohorts can defeat privacy even if the protocol messages are cryptographically protected.

Metadata leaks

The server may learn which clients participated, when they connected, and the aggregate for small cohorts.

Failure modes

- Small cohorts reveal individuals.
- Malicious clients poison the aggregate.
- Dropout handling leaks masks or values.
- Repeated aggregates enable differencing attacks.

Variants

- Single-server secure aggregation.
- Multi-server aggregation.
- Aggregation with differential privacy.

Concrete protocol families

Family or system	Typical role	Key differences and cautions
Bonawitz-style secure aggregation	Federated learning with client dropout	Pairwise masks cancel in aggregate; dropout handling and cohort size dominate privacy.
Prio / Prio+ style systems	Private telemetry with validity checks	Adds client-side proof or verification machinery so malformed values are harder to inject.

Family or system	Typical role	Key differences and cautions
Secret-shared aggregation	Multi-server private aggregation	Privacy depends on non-collusion or corruption threshold between helper servers.
Homomorphic-encryption aggregation	Encrypted sums with trustee or server decryption	Useful when client coordination is limited; key management and output leakage remain.
Differentially private aggregation	Aggregate release with noise	Not just cryptography; privacy budget and repeated releases are central.

Where it is used

- Federated analytics.
- Federated learning.
- Private telemetry.

Further reading

- Bonawitz et al., "Practical Secure Aggregation for Privacy-Preserving Machine Learning."
- Canetti, "Universally Composable Security; A New Paradigm for Cryptographic Protocols."

Multi-Party Computation

Goal

Multi-party computation (MPC) lets parties jointly compute a function over their inputs while keeping those inputs private under a stated adversary model.

Participants

Multiple parties with private inputs. Some protocols also use dealers or preprocessing parties.

Inputs and outputs

Each party supplies an input. The protocol reveals an output to designated parties.

Building blocks

- Secret sharing.
- Oblivious transfer.
- Oblivious pseudorandom functions in some specialized protocols.
- Commitments.
- Zero-knowledge proofs, in malicious-secure settings.

Security goals

- Input privacy.
- Correctness.
- Sometimes fairness or guaranteed output delivery.

Non-goals

- Hiding the output.
- Preventing inference from the output.
- Simplicity of deployment.

Threat model

MPC protocols differ for semi-honest, malicious, honest-majority, dishonest-majority, synchronous, and asynchronous settings.

Protocol sketch

Parties encode or share inputs, evaluate a circuit or arithmetic computation collaboratively, then reconstruct the allowed output.

Trust assumptions

Assumptions include corruption thresholds, network model, setup, and whether preprocessing is trusted or verifiable.

Post-quantum posture

Depends on the protocol and its building blocks. Information-theoretic MPC variants can avoid public-key assumptions in some settings, while many practical protocols rely on signatures, oblivious transfer, commitments, or channels whose posture must be checked separately.

Confidence model

Confidence is model-specific: honest-majority, dishonest-majority, threshold, semi-honest, or malicious. The page for a concrete MPC protocol should state the maximum corrupt set it tolerates and what happens on abort.

Metadata leaks

Participation, timing, circuit shape, aborts, and output values can leak information.

Failure modes

- Selecting a protocol for the wrong adversary model.
- Revealing too much through outputs.
- Ignoring aborts or fairness.
- Weak authentication between parties.

Variants

- Secret-sharing-based MPC.
- Garbled circuits.
- MPC with preprocessing.
- Threshold signing as a specialized use case.

Concrete protocol families

Family	Common setting	Typical role	Key differences and cautions
Yao garbled circuits	Two-party computation	Boolean-circuit MPC	Often efficient for two parties; oblivious transfer and malicious-security upgrades matter.
GMW	Multi-party Boolean circuits	General MPC with interactive rounds	Round complexity and network latency can dominate.
BGW	Honest-majority arithmetic MPC	Information-theoretic MPC in suitable settings	Requires honest majority and synchronous-style assumptions.

Family	Common setting	Typical role	Key differences and cautions
SPDZ-style protocols	Preprocessed arithmetic MPC	Dishonest-majority computation with offline preprocessing	Preprocessing generation and MAC checks are part of the confidence model.
MASCOT / OT-extension preprocessing	Practical malicious-secure preprocessing	Generating correlated randomness for SPDZ-like protocols	Relies on oblivious transfer and implementation-specific security choices.
Threshold signing protocols	FROST, threshold ECDSA, threshold BLS	Specialized MPC for signing	The signing protocol inherits both MPC threshold assumptions and signature-scheme assumptions.

Where it is used

- Threshold custody.
- Private analytics.
- Auctions.
- Collaborative risk scoring.
- Private set intersection variants.

See also Private Set Intersection, which can be built from MPC techniques or from more specialized protocol families.

Further reading

- Canetti, "Universally Composable Security; A New Paradigm for Cryptographic Protocols."
- Boneh and Shoup, "A Graduate Course in Applied Cryptography."

Mixnets

Goal

A mixnet breaks the link between message senders and outputs by batching, re-encrypting or transforming, and shuffling messages through one or more mix servers.

Participants

- Senders.
- Mix servers.
- Recipients or bulletin board.

Inputs and outputs

Senders submit messages. The mixnet outputs the same logical messages in a shuffled form.

Building blocks

- Public-key encryption.
- Verifiable shuffles.
- Commitments and proofs.

Security goals

- Sender-message unlinkability within an anonymity set.
- Verifiable correct shuffling in some designs.

Non-goals

- Perfect anonymity against global traffic analysis without batching assumptions.
- Protection if all mix servers collude.
- Coercion resistance by itself.

Threat model

The model must state how many mix servers may be corrupt and what the adversary observes about timing and network traffic.

Protocol sketch

Messages are collected into a batch, passed through mixes that transform and shuffle them, and then published or delivered.

Trust assumptions

Privacy often relies on at least one honest mix server and adequate batching.

Post-quantum posture

Depends on the encryption, signatures, and shuffle proofs used by the mixnet. A mixnet can have a one-honest-server privacy model while still relying on quantum-vulnerable public-key primitives.

Confidence model

The common confidence model is one-honest-party plus batching: privacy can hold if at least one mix server honestly shuffles and enough messages are mixed together. Verifiable mixnets add public verification for correct shuffling.

Metadata leaks

Batch size, timing, message size, and participation patterns can reveal users.

Failure modes

- Small batches.
- All mixes collude.
- Malformed shuffles.
- Side-channel leakage through message formats.

Variants

- Chaumian mixnets.
- Verifiable mixnets.
- Decryption mixnets.

Concrete protocols and packet formats

Family or system	Typical role	Key differences and cautions
Chaumian mixnets	Batched anonymous message delivery	Privacy depends on batching and at least one honest mix.
Verifiable shuffle mixnets	Elections and public bulletin boards	Adds proofs that shuffles are correct; proof system and public auditability matter.
Decryption mixnets	Encrypted ballots or messages decrypted through mixes	Mix servers transform and decrypt layers; trustee collusion breaks privacy.
Sphinx packet format	Anonymous messaging packets	Hides routing metadata inside fixed-format packets; timing analysis still matters.
Loopix-style mixnets	Continuous-time anonymous messaging	Adds delays and cover traffic; latency and traffic assumptions are part of the model.

Where it is used

- Electronic voting.
- Anonymous messaging.
- Privacy-preserving publication systems.

Further reading

- Chaum, "Untraceable Electronic Mail, Return Addresses, and Digital Pseudonyms."
- Benaloh, "Verifiable Secret-Ballot Elections."

Electronic Voting

Goal

Electronic voting systems aim to collect, protect, tally, and verify votes under demanding privacy, integrity, and coercion constraints.

Participants

- Voters.
- Election authority.
- Trustees or tally authorities.
- Observers and auditors.

Inputs and outputs

Voters input ballots. The system outputs a tally and audit evidence.

Building blocks

- Authentication or eligibility checks.
- Ballot encryption.
- Zero-knowledge proofs.
- Mixnets or homomorphic tallying.
- Threshold decryption.

Security goals

- Eligibility.
- Ballot secrecy.
- Verifiability.
- Integrity.
- Coercion resistance, in stronger systems.

Non-goals

No single cryptographic primitive makes a voting system safe. Operational procedures, usability, legal rules, and public auditability matter.

Threat model

Voting systems must consider malicious voters, corrupt authorities, compromised devices, coercers, network observers, and public trust.

Protocol sketch

Eligible voters cast protected ballots. Ballots are checked for validity, anonymized or aggregated, tallied, and audited.

Trust assumptions

Assumptions may include trustee honesty thresholds, device integrity, public bulletin boards, and audit processes.

Post-quantum posture

Depends on the election cryptography: voter authentication, ballot encryption, proofs, signatures, mixnets, and trustee key management may each have different posture. A voting system should not receive a single post-quantum label unless every relevant layer is classified.

Confidence model

Confidence usually comes from threshold trustees, public verifiability, eligibility authorities, client-device assumptions, and audit procedures. Ballot secrecy may be `t-of-n`, while integrity may come from public verification and challenge processes.

Metadata leaks

Timing, voter check-in, device identifiers, and small precinct totals can leak information.

Failure modes

- Validity checks leak votes.
- Receipts enable coercion.
- Malware changes ballots before encryption.
- Trustees collude.
- The audit trail is too complex for public confidence.

Variants

- Mixnet-based voting.
- Homomorphic tallying.
- End-to-end verifiable voting.
- Coercion-resistant voting protocols.

Concrete systems and protocol families

Family or system	Tally/privacy shape	Key differences and cautions
Helios-style voting	Public bulletin board, encrypted ballots, public verification	Useful for low-coercion settings; not coercion-resistant by default.
Mixnet tallying	Ballots are shuffled before decryption or publication	Stronger unlinkability when at least one mix is honest; shuffle proofs and batch size matter.

Family or system	Tally/privacy shape	Key differences and cautions
Homomorphic tallying	Encrypted ballots combine into an encrypted tally	Efficient for simple ballot formats; validity proofs must prevent malformed ballots.
Threshold decryption elections	Trustees jointly decrypt tally or ballots	Ballot secrecy depends on trustee threshold and key ceremony.
Coercion-resistant protocols	Receipt-free or fake-credential-resistant designs	Much harder system problem; usability and registration assumptions are critical.
DAO voting experiments	On-chain eligibility, nullifiers, ZK proofs, public tally	Ledger metadata, wallet funding, and coercion risks often dominate cryptography.

Where it is used

- Government elections in limited settings.
- Organization voting.
- DAO governance experiments.

Further reading

- Benaloh, "Verifiable Secret-Ballot Elections."
- Chaum, "Untraceable Electronic Mail, Return Addresses, and Digital Pseudonyms."
- Canetti, "Universally Composable Security; A New Paradigm for Cryptographic Protocols."

Design Patterns

Design Patterns Overview

Design patterns are recurring ways to compose cryptographic tools into system behavior.

Examples

- Anonymous membership proves eligibility without naming the user.
- Anti-double-use nullifiers prevent repeated anonymous actions.
- Private aggregation reveals totals without exposing individual values.
- Delayed reveal separates commitment time from opening time.
- Reveal only a function limits disclosure to a computed output.
- Make receipts useless reduces coercion and vote-buying risk.

Reading rule

A pattern is not a complete protocol. Treat it as a design shape that still needs assumptions, threat models, and implementation review.

Anonymous Membership

One-sentence intuition

Anonymous membership lets someone prove they belong to an eligible group without revealing which member they are.

Common building blocks

- Anonymous credentials.
- Merkle or accumulator membership proofs.
- Zero-knowledge proofs.
- Context binding.

What it does not provide

- Anti-double-use unless nullifiers or rate limits are added.
- Network anonymity.
- Coercion resistance.
- Privacy for very small groups.

Assumptions

The eligibility source, membership commitment, proof system, and verifier context binding must all match the intended group. Privacy also assumes the anonymity set is large enough and that presentations are not linkable through metadata.

Post-quantum posture

Not applicable to the pattern by itself. A concrete anonymous-membership system inherits posture from its credential scheme, membership proof, hash function, signatures, and transport layer.

Confidence model

Confidence usually depends on a trusted issuer or public membership set, holder-controlled secrets, public verification of membership proofs, and non-linking contexts.

Concrete compositions

Composition	Typical role	Main caution
Merkle membership plus zero-knowledge proof	Anonymous allowlist or group membership	The anonymity set is only the committed set, and stale roots can break eligibility.
Accumulator membership plus zero-knowledge proof	Compact anonymous membership with dynamic sets	Witness updates and accumulator setup must be part of the protocol.

Composition	Typical role	Main caution
BBS+ or CL anonymous credential	Attribute-based anonymous authorization	Issuer trust, revocation, and rare attributes can re-identify users.
Semaphore-style group membership	Anonymous signaling and one-action-per-group designs	Nullifier context design controls linkability and rate limits.

Failure modes

- The eligible set is too small.
- Membership data is stale or manipulable.
- Proofs are linkable across contexts.
- Metadata reveals the member.

Further reading

- Jan Camenisch and Anna Lysyanskaya, "An Efficient System for Non-transferable Anonymous Credentials with Optional Anonymity Revocation."
- Goldwasser, Micali, and Rackoff, "The Knowledge Complexity of Interactive Proof Systems."

Anti-Double-Use Nullifiers

One-sentence intuition

Anti-double-use nullifiers let a system reject repeated anonymous actions in the same context.

Pattern

1. Bind the action to a context.
2. Derive a public nullifier from a private secret and that context.
3. Prove the nullifier is well formed and eligible.
4. Reject the action if the nullifier was already used.

What it does not provide

- Eligibility by itself.
- Protection against stolen secrets.
- Privacy if contexts are reused badly.
- Coercion resistance.

Assumptions

The private secret must have enough entropy, the context must be domain-separated, and the system must check a public spent-nullifier set before accepting an action.

Post-quantum posture

Not applicable to the pattern by itself. A concrete nullifier design inherits posture from the hash function, proof system, credential scheme, and public registry mechanism.

Confidence model

Confidence comes from client-side secret control, deterministic context binding, public duplicate detection, and a proof that the nullifier is derived from an eligible secret.

Concrete compositions

Composition	Typical role	Main caution
Hash secret plus context	Simple one-use marker	Only safe when the secret has high entropy and the context is domain-separated.
ZK proof plus nullifier	Anonymous voting, airdrops, signaling	The proof must bind membership, context, and nullifier into one statement.
Credential serial number	Anonymous credential spending or presentation limits	Revocation and issuer linkability need separate treatment.

Composition	Typical role	Main caution
Epoch-scoped nullifier	Rate limits per time window or application	Epoch design controls whether users are linkable across periods.

Failure modes

- Reusing the same context links actions.
- Omitting domain separation.
- Storing side metadata that identifies the user.
- Accepting nullifiers without proving eligibility.

Related concepts

- Nullifiers

Further reading

- Nullifiers
- Anonymous membership

Private Aggregation

One-sentence intuition

Private aggregation reveals a combined result without revealing each participant's individual value.

Where it sits in the taxonomy

- Level: design pattern
- Parent category: design patterns
- Related concepts: homomorphic encryption, homomorphic commitments, MPC, secure aggregation

Problem it solves

Many systems need aggregate information: a vote total, a usage statistic, a risk score, or a sum of measurements. Private aggregation tries to compute that aggregate while keeping each input hidden.

Mental model

Participants place values into sealed envelopes that can be combined. The system opens only the total, not the individual envelopes.

Minimal example

In a private poll, each voter submits an encrypted vote. The system combines encrypted votes and decrypts only the final tally.

If participant `i` holds a private value x_i, the system may reveal only the aggregate:

$$T = \sum_{i=1}^{n} x_i$$

The privacy goal is to reveal T without revealing the individual values $x_1, \ldots, x_n$.

Approaches

Approach	Useful when	Main risk
Homomorphic encryption	A public aggregator should combine ciphertexts	Key management and invalid inputs
Homomorphic commitments	Values need binding plus additive structure	Requires proofs that values are valid
Multi-party computation (MPC)	No single party should see inputs	Assumptions about parties and availability
Secure aggregation	Many clients report statistics	Dropout, malicious clients, and small groups

Security properties

- Individual input privacy, under the chosen model.
- Aggregate correctness, if inputs are valid and the protocol completes.
- Limited disclosure of the final aggregate.

What it does not provide

- Privacy for very small groups.
- Protection against differencing attacks across repeated queries.
- Proof that inputs are valid unless validity checks are added.
- Protection against metadata leaks.
- Coercion resistance in voting contexts.

Assumptions

Assumptions vary by construction: encryption keys, threshold decryption, honest-majority assumptions, dropout handling, authenticated participants, and zero-knowledge proofs for input validity may all be required.

Post-quantum posture

Not applicable to the pattern by itself. The posture is inherited from the aggregation mechanism, authentication, proof system, transport, and key-management layers.

Confidence model

Confidence may come from threshold decryption, non-colluding helper servers, honest-majority MPC, or public verification of encrypted inputs. The page for a concrete design should state which model is being used.

Use cases

- Voting and polling.
- Private telemetry.
- Private statistics.
- Federated analytics.
- Confidential financial sums.

Composition patterns

Private aggregation is often combined with range proofs, membership proofs, rate limits, threshold decryption, or differential privacy.

Concrete compositions

Composition	Typical role	Main caution
Paillier-style or additive homomorphic encryption tally	Simple encrypted sums in legacy or specialized systems	Quantum-vulnerable and key-management-heavy; validity proofs are still required.
Lattice HE tally	Post-quantum-oriented encrypted aggregation	Parameter choice and output leakage dominate practical risk.
Bonawitz-style secure aggregation	Federated learning and telemetry	Dropout handling, cohort size, and malicious updates are the main failure points.
Prio-style private telemetry	Aggregate statistics with validity checks	Requires a validation mechanism so clients cannot poison aggregates.
MPC-based aggregation	Multi-server or multi-party analytics	Collusion threshold and abort behavior must be explicit.

Failure modes and anti-patterns

- Aggregates over tiny groups reveal individuals.
- Repeated aggregates allow differencing attacks.
- Malicious users submit invalid or extreme values.
- A decryptor quorum can collude or lose keys.
- Metadata reveals who participated and when.

A differencing attack appears when two aggregates differ by one participant:

$$x_j = T(S \cup \{j\}) - T(S)$$

This is why query design, cohort size, and repeated releases matter.

Maturity and deployment

The pattern is mature, but concrete systems range from well deployed to experimental depending on scale, threat model, and implementation.

Related concepts

- Homomorphic encryption
- Secure aggregation
- Private set intersection
- Private DAO voting

Further reading

- Bonawitz et al., "Practical Secure Aggregation for Privacy-Preserving Machine Learning."
- Gentry, "Fully Homomorphic Encryption Using Ideal Lattices."
- Benaloh, "Verifiable Secret-Ballot Elections."

Delayed Reveal

One-sentence intuition

Delayed reveal fixes information at one time and discloses it later.

Common building blocks

- Commitments.
- Timelocks or VDFs.
- Public bulletin boards.
- Opening deadlines and dispute rules.

What it does not provide

- Confidentiality after opening.
- Fairness if parties can abort without penalty.
- Authentication unless submissions are bound to identities or credentials.

Assumptions

The commitment or delay mechanism must bind the initial value, opening rules must be unambiguous, and the system must define what happens when a party refuses or misses the reveal phase.

Post-quantum posture

Not applicable to the pattern by itself. A concrete design inherits posture from its commitments, timelocks, signatures, and public bulletin board.

Confidence model

Confidence may come from mathematical binding, public-verifiability of openings, or external-timing assumptions when VDFs or timelocks are used.

Concrete compositions

Composition	Typical role	Main caution
Hash commit-reveal	Lotteries, auctions, simple delayed disclosure	Weak randomness reveals low-entropy committed values.
Pedersen commit-reveal	Numeric hidden values with later opening	Quantum-vulnerable; randomness and generator setup matter.
Timelock puzzle reveal	Delay without relying only on a human opener	Delay assumptions and hardware advantage must be modeled.
VDF-assisted reveal	Publicly verifiable delayed output	VDF setup and denial-of-service handling are part of the design.

Failure modes

- Selective aborts.
- Weak randomness in commitments.
- Ambiguous opening formats.
- No rule for missed deadlines.

Further reading

- Commitments
- Timelocks and VDFs

Reveal Only a Function

One-sentence intuition

Reveal only a function means exposing a computed result while keeping the underlying inputs hidden.

Common building blocks

- Homomorphic encryption.
- Functional encryption.
- Multi-party computation.
- Zero-knowledge proofs for input validity.

What it does not provide

- Protection if the function output is too revealing.
- Privacy across repeated queries without leakage analysis.
- Correctness unless inputs and computation are verified.

Assumptions

The allowed function must be chosen carefully, repeated queries must be controlled, and the underlying primitive or protocol must enforce that only the intended function result is revealed.

Post-quantum posture

Not applicable to the pattern by itself. A concrete system inherits posture from functional encryption, homomorphic encryption, multi-party computation, proof systems, and authentication.

Confidence model

Confidence depends on the mechanism: a key authority for functional encryption, threshold or honest-party assumptions for multi-party computation, or mathematical assumptions and key control for homomorphic encryption.

Concrete compositions

Composition	Revealed value	Main caution
Homomorphic encryption plus threshold decryption	Aggregate or limited computation result	Trustees and output leakage define the confidence model.
MPC computation	Function output from private inputs	Abort behavior and the revealed output can leak sensitive information.
Functional encryption	Function value authorized by a function key	Key issuer trust and repeated-query leakage are central.

Composition	Revealed value	Main caution
ZK proof plus public computation	Proof that a hidden input satisfies a function predicate	The predicate may still reveal sensitive facts.

Failure modes

- Differencing attacks across multiple outputs.
- Functions that encode private values directly.
- Missing access control around who can ask which function.

Further reading

- Functional encryption
- Multi-party computation

Make Receipts Useless

One-sentence intuition

This pattern tries to stop users from proving to a coercer how they acted.

Problem it solves

Some systems need more than privacy from observers. They need to prevent a participant from producing convincing evidence of their own action.

Common approaches

- Deniable or fakeable transcripts.
- Re-voting where only the last vote counts.
- Controlled credential recovery or revocation.
- Mixnets or tallying designs that avoid public vote receipts.

What it does not provide

- Protection against all real-world coercion.
- Safety on compromised devices.
- A simple add-on to ordinary privacy systems.

Assumptions

The system must define the coercion model, control what public evidence is created, and account for user interfaces, logs, screenshots, credential recovery, and repeated participation.

Post-quantum posture

Not applicable to the pattern by itself. Concrete posture depends on the voting, credential, encryption, proof, and tallying mechanisms used.

Confidence model

Confidence usually combines public verifiability for tally integrity with protocol features that make user-held evidence deniable, fakeable, revocable, or superseded by later actions.

Concrete compositions

Composition	Typical role	Main caution
Re-voting with last vote counts	Reducing value of early coerced receipts	Does not help if coercion happens after the final opportunity to vote.

Composition	Typical role	Main caution
Fakeable credential or transcript	Let users simulate evidence for any choice	Hard to make convincing without weakening auditability.
Mixnet tallying without per-voter receipts	Hide ballot-to-voter linkage	Device compromise or check-in metadata can still create receipts.
Coercion-resistant credential recovery	Let voters invalidate coerced credentials	Registration and recovery channels become part of the threat model.

Failure modes

- User interfaces expose receipts.
- Logs or screenshots become proofs.
- Coercers demand credentials before the action.
- Small groups make choices inferable from outcomes.

Further reading

- Electronic voting
- Coercion-resistant voting

Case Studies

Case Studies Overview

Case studies show how multiple concepts interact in system designs. They are maps, not deployment guides.

Current studies

- Private DAO voting.
- Coercion-resistant voting.
- Anonymous airdrops.

Reading rule

Track goals, non-goals, building blocks, leaks, and maturity warnings separately.

Private DAO Voting

Overview

This case study sketches a private DAO voting system as a composition exercise. It is not a deployable protocol.

Goals

- Eligible members can vote.
- Ballots remain private until, or unless, tally disclosure is intended.
- A member cannot vote twice in the same election.
- Invalid ballots are rejected.
- The final tally is verifiable.

Non-goals

- Full coercion resistance.
- Protection against all traffic analysis.
- Production deployment guidance.
- Governance security outside the voting protocol.
- Legal or regulatory compliance.

Possible building blocks

Requirement	Possible building block
Anonymous eligibility	anonymous credentials or membership proofs
Anti-double-vote protection	context-specific nullifiers
Ballot secrecy	public-key encryption or threshold encryption
Valid ballot proofs	zero-knowledge proofs
Private tallying	homomorphic encryption, MPC, or threshold decryption
Delayed reveal	timelock pattern or governance-defined opening phase

Concrete composition options

Requirement	Concrete options	Main caution
Eligibility set	Merkle tree, sparse Merkle tree, accumulator, anonymous credential issuer	Set construction affects anonymity, updates, and revocation.
Nullifier	Hash nullifier, Semaphore-style nullifier, credential serial number	Context binding controls cross-election linkability.
Ballot privacy	ElGamal-style encryption, threshold encryption, homomorphic encryption	Many classical options are quantum-vulnerable; trustees and keys dominate confidence.

Requirement	Concrete options	Main caution
Validity proof	Groth16, PLONK-style proof, STARK, Bulletproof-style range proof	Proof must bind ballot, election context, nullifier, and eligibility.
Tally	Homomorphic tally, mixnet tally, MPC tally	The tally method determines trustee, batching, and audit assumptions.

Simple architecture

1. The DAO publishes an election context, eligible voter set, and ballot rules.
2. Each eligible voter derives a context-specific nullifier.
3. The voter encrypts or commits to a ballot.
4. The voter submits a zero-knowledge proof that the ballot is valid and the nullifier is derived from an eligible secret.
5. The system rejects duplicate nullifiers.
6. The tally is computed privately and revealed with verification data.

Privacy leaks

- Submission timing can correlate voters with ballots.
- Wallet funding and transaction fees can identify voters.
- Small voter groups can reveal preferences from the final tally.
- Reused contexts or poor domain separation can link votes across elections.
- Public discussion, delegation, or governance forums can leak intent.

Post-quantum posture

Depends on every selected building block: credential signatures, membership proofs, nullifiers, ballot encryption, validity proofs, and tallying. A DAO voting design should be treated as post-quantum only if each layer has been classified.

Confidence model

A typical design combines several confidence models: trusted or semi-trusted eligibility source, holder secrets for anonymous voting, public nullifier registry for non-reuse, proof-system soundness for ballot validity, and threshold trustees or MPC participants for tally privacy.

Coercion limitations

Ballot secrecy is not the same as coercion resistance. A voter may be pressured to reveal credentials, prove how they voted, vote under observation, or sell access to a voting key. Coercion-resistant voting needs additional protocol and operational design.

Maturity warning

> **WARNING**
>
> Private DAO voting combines fast-moving privacy tooling with governance incentives and public ledgers. Treat this as a design map, not an implementation recommendation.

Failure modes

- Eligibility source is centralized or manipulable.
- Nullifier construction links voters across elections.
- Invalid encrypted ballots pass because validity proofs are incomplete.
- Tally authorities collude or lose decryption shares.
- User interfaces accidentally reveal votes or create receipts.
- Governance rules do not define aborts, challenges, or disputed tallies.

Related concepts

- Nullifiers
- Zero-knowledge proofs
- Private aggregation
- Coercion-resistant voting

Further reading

- Benaloh, "Verifiable Secret-Ballot Elections."
- Goldwasser, Micali, and Rackoff, "The Knowledge Complexity of Interactive Proof Systems."
- Bonawitz et al., "Practical Secure Aggregation for Privacy-Preserving Machine Learning."

Coercion-Resistant Voting

Overview

Coercion-resistant voting aims to prevent a voter from proving how they voted, even if a coercer pressures them.

Goals

- Ballot secrecy.
- Receipt-freeness.
- Resistance to forced abstention or forced choice, depending on the model.
- Verifiable tallying.

Non-goals

- Solving all physical coercion.
- Protecting compromised voter devices by cryptography alone.
- Simple deployment.

Building blocks

- Anonymous credentials.
- Re-voting or credential recovery.
- Mixnets or homomorphic tallying.
- Zero-knowledge proofs.
- Careful user experience and operational procedures.

Concrete composition options

Requirement	Concrete options	Main caution
Eligibility	Anonymous credentials, recovery credentials, private registration lists	Registration can itself create coercion and tracking channels.
Ballot privacy	Mixnet tallying, homomorphic tallying, threshold decryption	The tally style changes trustee and metadata assumptions.
Receipt resistance	Re-voting, fakeable transcripts, credential invalidation	Must match the coercion timing model.
Validity and audit	ZK ballot-validity proofs, public bulletin board, verifiable shuffles	Public audit data must not become a voter-held receipt.
Trustee model	Threshold trustees, distributed key generation, public ceremony	Trustee collusion and key loss are system-level failures.

Privacy leaks

Timing, small groups, device compromise, and social pressure can defeat formal privacy claims.

Post-quantum posture

Depends on voter authentication, credentials, ballot encryption, proof systems, mixnets or tallying, signatures, and trustee key management. Coercion resistance is a system property and does not receive a single post-quantum label unless every cryptographic layer is classified.

Confidence model

Confidence usually combines public verifiability, threshold trustees, trusted eligibility processes, client-side secrecy, and operational controls that make receipts unavailable, fakeable, or superseded by later actions.

Maturity warning

Coercion resistance is a demanding system property. Treat any simple claim of coercion resistance with skepticism unless the threat model is precise.

Failure modes

- The voter interface creates a receipt.
- Device compromise observes or changes the vote.
- Coercers demand credentials before voting.
- Small groups or public tallies make votes inferable.
- Re-voting or recovery rules are too hard for real voters to use.

Related concepts

- Electronic voting
- Make receipts useless
- Mixnets

Further reading

- Benaloh, "Verifiable Secret-Ballot Elections."
- Electronic voting

Anonymous Airdrop

Overview

An anonymous airdrop lets eligible users claim once without publicly linking the claim to their original identity.

Goals

- Eligibility.
- One claim per eligible user.
- Claim privacy.
- Public auditability of the spent claim set.

Non-goals

- Network anonymity.
- Protection from identity leaks during funding or withdrawal.
- Sybil resistance beyond the eligibility source.

Possible building blocks

- Eligibility set commitment.
- Membership proof.
- Nullifier.
- Zero-knowledge proof.
- Private withdrawal or shielding mechanism.

Concrete composition options

Requirement	Concrete options	Main caution
Eligibility commitment	Merkle tree, sparse Merkle tree, accumulator	The set root must be authenticated and deduplicated.
Membership proof	Merkle proof inside ZK, accumulator witness, credential presentation	Proof choice determines setup, posture, and witness update risk.
One-claim enforcement	Hash nullifier, ZK-derived nullifier, credential serial number	Nullifier context must be campaign-specific.
Claim privacy	Shielded pool, delayed withdrawal, mixnet-style relay	Public ledger metadata can still identify claimants.
Validity proof	Groth16, PLONK-style, STARK-style proof	The proof must include eligibility, nullifier correctness, and claim rules.

Privacy leaks

Claim timing, gas funding, wallet reuse, and exchange withdrawals can identify claimants.

Post-quantum posture

Depends on the selected membership proof, nullifier construction, signatures, shielding mechanism, and transaction layer. A hash-based nullifier does not make the whole airdrop post-quantum if the credential or proof system is quantum-vulnerable.

Confidence model

Confidence combines a trusted or publicly auditable eligibility source, holder-controlled secrets, public spent-nullifier checks, proof-system soundness, and operational controls around funding and withdrawal privacy.

Failure modes

- Eligibility set contains duplicates.
- Nullifiers are linkable across campaigns.
- Claims reveal wallet funding patterns.
- The proof omits a required context.

Related concepts

- Nullifiers
- Anonymous membership
- Anti-double-use nullifiers

Further reading

- Nullifiers
- Membership proofs

Glossary

Glossary

Authenticated encryption

A symmetric encryption interface that provides plaintext confidentiality and lets receivers reject modified ciphertexts, often while authenticating public associated data.

Related: Authenticated encryption, Symmetric encryption, Message authentication codes.

Anonymity

The property that an actor is hidden within a set of possible actors.

Related: Security goals, Anonymous membership, Mixnets.

Binding

For commitments, the property that a committer cannot open one commitment to two different values.

Related: Commitments, Pedersen commitments, Homomorphic commitments.

Commitment

A primitive that hides a value while binding the committer to that value for a later opening.

Related: Commitments, Delayed reveal, Range proofs.

Completeness

For proof systems, honest proofs for true statements should verify.

Related: Zero-knowledge proofs, SNARKs, STARKs, and Bulletproofs.

Confidence model

The source of confidence for a cryptographic guarantee, such as a hardness assumption, public verification, a trusted issuer, one honest party, or a threshold of honest parties.

Related: Confidence models, Assumptions, Trusted setup.

Discrete logarithm

A mathematical problem where exponentiation is easy in a selected group but recovering the exponent should be hard.

Related: Discrete logarithm, Digital signatures, Pedersen commitments.

Factoring

The problem of recovering prime factors from a composite number.

Related: Factoring and RSA, Public-key encryption, Post-quantum posture.

Hiding

For commitments, the property that the commitment does not reveal the committed value before opening.

Related: Commitments, Pedersen commitments.

Lattice

A structured high-dimensional grid used as the substrate for many post-quantum schemes.

Related: Lattices, Homomorphic encryption, Key encapsulation and exchange.

Metadata leak

Information revealed outside the protected cryptographic payload, such as timing, size, participation, or public inputs.

Related: Metadata leakage, Composability, Threat-model checklist.

Multi-party computation

A protocol family where parties jointly compute a function without revealing their private inputs beyond the output.

Related: MPC, Secure aggregation, Reveal only a function.

Nullifier

A public value used to detect repeated anonymous actions within a context.

Related: Nullifiers, Anti-double-use nullifiers, Anonymous airdrop.

Oblivious pseudorandom function

A two-party protocol where a client learns a keyed pseudorandom function output for its private input without learning the key and without revealing the input to the server.

Related: Oblivious pseudorandom functions, Private set intersection, Discrete logarithm.

Pairing

A special map between algebraic groups that makes some hidden exponent relationships publicly checkable.

Related: Pairings, SNARKs, STARKs, and Bulletproofs, Trusted setup.

Post-quantum posture

An editorial classification of whether a construction is quantum-vulnerable, plausibly post-quantum, dependent on instantiation, unknown, or not applicable.

Related: Post-quantum posture, Lattices, Discrete logarithm.

Private set intersection

A protocol family that lets parties learn the overlap, size of overlap, or approved function of their private sets while limiting disclosure about non-matching elements.

Related: Private set intersection, Oblivious pseudorandom functions, MPC.

Random oracle model

An idealized proof model that treats a hash function as a public random function.

Related: Random oracle model, Hash functions, Zero-knowledge proofs.

Receipt-freeness

A property where a participant cannot create convincing evidence of how they acted.

Related: Security goals, Make receipts useless, Coercion-resistant voting.

Soundness

For proof systems, false statements should not verify except with negligible probability.

Related: Zero-knowledge proofs, Range proofs, Membership proofs.

Trusted setup

A parameter-generation process whose hidden trapdoor must not survive for the system's claims to hold.

Related: Trusted setup, SNARKs, STARKs, and Bulletproofs, Confidence models.

Unlinkability

The property that two actions cannot be linked to the same actor under the stated model.

Related: Security goals, Anonymous credentials, Mixnets.

Witness

Private information that makes a public proof statement true.

Related: Zero-knowledge proofs, Range proofs, Membership proofs.

Zero-knowledge

The property that a proof reveals no information beyond the truth of the statement, under the proof system's model.

Related: Zero-knowledge proofs, SNARKs, STARKs, and Bulletproofs, Random oracle model.

Appendices

Reading List

This reading list mirrors the structured reference registry in `book/data/references.yml`.

General applied cryptography

- Boneh and Shoup, "A Graduate Course in Applied Cryptography."
- Katz and Lindell, "Introduction to Modern Cryptography."
- David Wong, "Real-World Cryptography."
- Jean-Philippe Aumasson, "Serious Cryptography."

Symmetric encryption and authentication

- RFC 5116, "An Interface and Algorithms for Authenticated Encryption."
- Rogaway, "Authenticated-Encryption with Associated-Data."
- RFC 8439, "ChaCha20 and Poly1305 for IETF Protocols."
- NIST SP 800-38D, "Recommendation for Block Cipher Modes of Operation: Galois/Counter Mode (GCM) and GMAC."

Zero-knowledge proofs

- Goldwasser, Micali, and Rackoff, "The Knowledge Complexity of Interactive Proof Systems."
- Ben-Sasson et al., "Scalable, transparent, and post-quantum secure computational integrity."
- Bünz et al., "Bulletproofs: Short Proofs for Confidential Transactions and More."

Electronic voting

- Benaloh, "Verifiable Secret-Ballot Elections."
- Chaum, "Untraceable Electronic Mail, Return Addresses, and Digital Pseudonyms."

MPC and secure aggregation

- Canetti, "Universally Composable Security; A New Paradigm for Cryptographic Protocols."
- Bonawitz et al., "Practical Secure Aggregation for Privacy-Preserving Machine Learning."
- Jarecki and Liu, "Efficient Oblivious Pseudorandom Function with Applications to Adaptive OT and Secure Computation of Set Intersection."
- RFC 9497, "Oblivious Pseudorandom Functions (OPRFs) Using Prime-Order Groups."
- Freedman, Nissim, and Pinkas, "Efficient Private Matching and Set Intersection."

Functional encryption

- Boneh, Sahai, and Waters, "Functional Encryption: Definitions and Challenges."

VDFs and timelock encryption

- Boneh et al., "Verifiable Delay Functions."
- Wesolowski, "Efficient Verifiable Delay Functions."

Notation

This appendix records lightweight notation used across the atlas.

Notation	Meaning
`m`	Message or value
`r`	Randomness
`C`	Commitment
`pk`	Public key
`sk`	Secret key
`Hash(x)`	Hash of input `x`
`context`	Domain-separated application or protocol context

Conventions

Examples are conceptual unless a page explicitly states that it is implementation-oriented.

Maturity Scale

Maturity labels describe deployment confidence at a high level. They are not security guarantees.

Label	Meaning
deployed	Used in production systems with meaningful operational history
mature	Well studied and commonly implemented, though maybe specialized
emerging	Actively used or piloted, but trade-offs are still settling
research	Mostly evaluated in papers, prototypes, or limited settings
theoretical	Mainly conceptual or impractical today
not-applicable	The label does not apply to this page

Rule

Maturity should be evaluated for a concrete construction and implementation, not only for a broad topic name.

Editorial Maturity and Coverage

This appendix states what "current" means for the atlas and which important concepts are still missing or shallow.

Status model

Status	Meaning in this book
current	Reviewed against the atlas template, taxonomy, caveat, post-quantum, confidence-model, and source expectations for the current scope. It does not mean exhaustive or production guidance.
needs-review	Known to be stale, shallow, or in a fast-moving area that needs a focused update before readers should treat it as current.
outdated	Contains claims that are known to be superseded or misleading.
draft	Structurally incomplete or not yet reviewed against the atlas editorial checklist.

Current maturity assessment

The taxonomy is good for the book's mission. It separates goals, assumptions, primitives, proof systems, protocols, systems, and design patterns, which prevents the common mistake of treating a named tool as a complete system design.

The recent "concrete instance" layer is the right way to place algorithms and schemes. It avoids turning names such as `AES-GCM`, `Ed25519`, `Groth16`, or `TLS 1.3` into a ninth peer taxonomy level. Those names should instead be described as instances of a parent concept, with `instance_of` metadata if the atlas later makes scheme coverage machine-readable.

One taxonomy tension remains: some objects are both building blocks and interactive protocols. Examples include oblivious transfer and oblivious pseudorandom functions. The current rule is:

- use `protocol` when the guarantee comes from interaction between parties;
- cross-link it as a building block where it is composed into MPC, PSI, credentials, or token systems;
- avoid creating a separate "protocol primitive" level unless the atlas later needs a more formal ontology.

Coverage added in this pass

Area	Added concept	Why it matters
Basic primitives	Authenticated encryption	Modern systems usually need confidentiality and integrity together; bare encryption is easy to misuse.
Protocols	Oblivious pseudorandom functions	OPRFs are a common building block for password hardening, tokens, credentials, and PSI.
Protocols	Private set intersection	PSI is one of the core privacy-preserving computation protocols missing from the initial protocol set.

Important missing concepts

These are the highest-value gaps for future expansion.

Taxonomy area	Missing or shallow concepts	Why they matter
Security goals	verifiability, auditability, accountability, forward secrecy, deniability	Many systems claim "privacy" while relying on public audit or transcript properties that need precise goals.
Assumptions and substrates	elliptic curves as a substrate, finite-field groups, code-based assumptions, hash-to-curve, common reference strings, standard-model vs random-oracle distinctions	Current assumption pages cover the main families, but concrete curve/group/setup choices deserve clearer treatment.
Basic primitives	pseudorandom functions, pseudorandom permutations, password hashing, nonce-misuse-resistant encryption, key-committing encryption	These explain everyday engineering risks that recur across protocols.
Structured primitives	verifiable random functions, blind signatures, polynomial commitments, vector commitments, verifiable encryption, e-cash primitives	These are heavily used in blockchain, credentials, rollups, and private payments.
Proof systems	polynomial commitments, FRI, folding schemes, recursive proofs, lookup arguments, sumcheck, arithmetization	The proof-system overview is useful, but ZK engineering needs more concept pages below the family level.
Protocols	oblivious transfer, private information retrieval, password-authenticated key exchange, secure channels, anonymous tokens, blind-signature credentials, private payments	These are common in deployed privacy systems and would strengthen the bridge from primitives to systems.
Systems	private payments, ZK rollups, secure messaging, privacy-preserving identity wallets, encrypted mempools, private machine-learning analytics	The current system coverage is privacy/voting-heavy; broader deployed systems would make the atlas more balanced.
Design patterns	encrypt-then-prove, threshold issuance, revocation with privacy, domain separation, transcript binding, key rotation and migration	These recurring engineering patterns are where many real systems fail.

Source-depth gaps

The source registry now covers the main historical and standards references for existing pages, but depth is uneven.

Prioritize deeper sources for:

- post-quantum migration and algorithm status;
- concrete ZK proof-system families and polynomial commitments;
- threshold signing and distributed key generation;
- anonymous credentials, revocation, and selective disclosure;
- private payments and ledger metadata leakage;
- secure-channel protocols such as TLS 1.3, HPKE, Noise, Signal X3DH, and Double Ratchet.

Recommended next content sequence

1. Add Oblivious Transfer as a protocol page and cross-link it from MPC and PSI.
2. Add Secure Channels covering TLS 1.3, HPKE, Noise, and Signal at the protocol-suite level.
3. Add Polynomial Commitments and Vector Commitments before expanding ZK rollup coverage.
4. Add Verifiable Random Functions and Blind Signatures as structured primitives used by randomness, credentials, and private payments.
5. Add Private Payments as a system page after the blind-signature and accumulator material is stronger.

Maintenance rule

For fast-moving areas such as post-quantum cryptography, zero-knowledge proof systems, fully homomorphic encryption, MPC frameworks, anonymous credentials, and blockchain privacy, re-check source freshness at least every six months. If a page has not been reviewed in that window, mark it `needs-review` rather than leaving it `current`.

Bottom line

The taxonomy is sound. The atlas is now more mature as an educational map, but it is not complete. The biggest remaining gap is not the top-level taxonomy; it is deeper coverage of concrete instances, protocol suites, and system case studies.

Comparison Matrices

These matrices are editorial shortcuts. They compare where to look first, not which construction to deploy.

Basic primitives

Primitive	Primary goal	Does not provide	Common risk
Hash functions	Fixed-length digest with preimage and collision resistance	Encryption, authentication, hiding low-entropy secrets	Missing domain separation or obsolete algorithms
Symmetric encryption	Confidentiality under a shared secret key	Key distribution or authentication by itself	Nonce misuse or unauthenticated ciphertexts
MACs	Shared-key integrity and authenticity	Public verifiability or confidentiality	Replay, key reuse, timing-leaky comparisons
Digital signatures	Public verifiability of message origin and integrity	Confidentiality or signer understanding	Ambiguous signing contexts or nonce reuse
Public-key encryption	Recipient-controlled confidentiality	Sender authentication or metadata privacy	Key misbinding or unsafe raw encryption
Secret sharing	Threshold reconstruction and confidentiality below threshold	Share authentication or governance	Correlated share custody or unclear recovery rules

Privacy protocols

Protocol	Main privacy goal	Confidence model	Metadata leaks
Anonymous credentials	Selective disclosure or unlinkable authorization	Trusted issuer plus holder secret	Issuer-verifier collusion, rare attributes, revocation checks
Nullifiers	One anonymous action per context	Holder secret plus public duplicate registry	Timing, transaction fees, reused contexts
Secure aggregation	Aggregate-only disclosure	Honest-majority or non-collusion depending on protocol	Participation, dropout, cohort size
MPC	Joint computation without revealing inputs	Adversary threshold and abort model	Participant set, circuit shape, aborts
Mixnets	Sender-recipient unlinkability through shuffling	At least one honest mix	Timing, batch membership, message size

Systems and patterns

Item	Level	Primary goal	Deployment caution
Anonymous membership	Design pattern	Prove eligibility without revealing member identity	Needs anti-double-use design when repeated action matters
Private aggregation	Design pattern	Reveal aggregate rather than individual inputs	Small groups and differencing attacks dominate many failures

Item	Level	Primary goal	Deployment caution
Delayed reveal	Design pattern	Commit now and open later	Define deadlines, challenges, and abort handling
Private DAO voting	System	Private eligible voting with public tally confidence	Not coercion-resistant by default
Anonymous airdrop	System	One private claim per eligible user	Public ledger metadata can defeat cryptographic anonymity

Data files

The machine-readable versions live in:

- `book/data/comparison-matrices/proof-systems.yml`
- `book/data/comparison-matrices/primitives.yml`
- `book/data/comparison-matrices/privacy-protocols.yml`
- `book/data/comparison-matrices/system-patterns.yml`

Concrete Algorithms and Schemes

This appendix indexes concrete algorithms, schemes, parameter families, and named protocol suites that are surfaced on the concept pages they instantiate.

Where algorithms sit in the taxonomy

Concrete algorithms inherit the taxonomy level of the concept they instantiate. They are not a separate top-level level; they sit in a cross-cutting instance layer attached to a parent concept.

Use this mental format:

```
Concrete instance of: [parent concept]
```

Examples: Ed25519 is a concrete instance of digital signatures, Groth16 is a concrete instance of SNARK-style proof systems, and TLS 1.3 is a concrete instance of secure-channel protocols.

Example	Taxonomy placement	Why
SHA-256	Basic primitive: hash functions	It is a concrete hash algorithm.
AES-GCM	Basic primitive: symmetric encryption / authenticated encryption	It is a concrete encryption mode and authentication composition.
HMAC-SHA-256	Basic primitive: message authentication codes	It is a concrete MAC construction.
Ed25519	Basic primitive: digital signatures	It is a concrete signature scheme.
X25519	Basic primitive: key exchange	It is a concrete Diffie-Hellman function over Curve25519.
ML-KEM	Basic primitive: key encapsulation	It is a concrete post-quantum key-encapsulation mechanism.
Groth16	Proof system	It is a concrete succinct proof system.
Pedersen commitment	Basic primitive: commitments	It is a concrete commitment scheme.
KZG commitment	Structured primitive: polynomial/vector commitment	It is a concrete commitment scheme with pairing and setup assumptions.

This distinction matters because a name like `SHA-256` does not explain its goal, assumptions, misuse cases, or composition role. The concept page should explain the primitive; the instance entry should explain the concrete trade-offs.

Why most algorithms are not first-class pages

The book is organized by concepts, not as an algorithm catalog. That keeps the taxonomy readable, but readers still encounter names such as `SHA-256`, `AES-GCM`, `Ed25519`, `X25519`, `BLS12-381`, `Groth16`, or `ML-KEM` before they understand where those names sit.

For that reason, concrete algorithms should usually appear first as comparison tables on their parent concept pages. A dedicated page is useful only when the named scheme has distinct assumptions,

failure modes, deployment status, or composition risks that would overload the parent page.

If this appendix later becomes machine-readable, use an `instance_of` relationship rather than a new `level` value. That preserves the main taxonomy while allowing filters such as "show concrete instances of digital signatures" or "show post-quantum instances of key encapsulation".

Concrete algorithms should be added when they do at least one of the following:

- appear frequently in real systems;
- change the trust assumptions or post-quantum posture;
- have important misuse hazards;
- are common in blockchain, zero-knowledge, privacy, or applied security systems;
- are legacy names readers still need to recognize.

Inclusion backlog by category

Assumptions and substrates

These are mostly parameter families, groups, curves, or problem families rather than standalone algorithms.

Area	Include first	Include later or mention as caution
Finite-field discrete logarithm	FFDHE groups, safe-prime groups	Small or custom groups
Elliptic curves	P-256, P-384, Curve25519, Curve448, secp256k1	P-521, legacy binary curves
Pairing-friendly curves	BLS12-381, BN254	BW6 curves, MNT curves
RSA substrate	RSA modulus sizes, public exponent conventions	Multi-prime RSA, raw RSA
Lattice assumptions	Learning with errors (LWE), module-LWE, short integer solution (SIS), module-SIS, NTRU lattices	Ring-LWE variants, parameter-set caveats
Hash-to-curve	RFC 9380 suites	Ad hoc hash-to-curve mappings

Basic primitives

Primitive page	Include first	Include later or mention as caution
Hash functions	SHA-256, SHA-384, SHA-512, SHA3-256, SHAKE128, SHAKE256, BLAKE2, BLAKE3	SHA-1 and MD5 as broken or legacy context
ZK-friendly hashes	Poseidon, Rescue, MiMC, Griffin	Pedersen hash, domain-specific circuit costs
Symmetric encryption and AEAD	AES-GCM, ChaCha20-Poly1305, XChaCha20-Poly1305, AES-GCM-SIV, AES-SIV	AES-CBC as legacy; AES-ECB as an anti-pattern
Block ciphers	AES-128, AES-192, AES-256	DES and 3DES as legacy context
Message authentication codes	HMAC-SHA-256, HMAC-SHA-512, CMAC-AES, GMAC, Poly1305, KMAC	CBC-MAC misuse outside its narrow setting

Primitive page	Include first	Include later or mention as caution
Key derivation and password hashing	HKDF, PBKDF2, scrypt, Argon2id	Raw hashes as password storage anti-patterns
Randomness and nonces	HMAC_DRBG, Hash_DRBG, CTR_DRBG, ChaCha20-based CSPRNGs, operating-system CSPRNG interfaces	Reused nonces, userland entropy mixers
Commitments	Hash commitments, Pedersen commitments, Merkle commitments	Low-entropy committed values without salt or hiding
Digital signatures	Ed25519, Ed448, ECDSA P-256, ECDSA secp256k1, RSA-PSS, Schnorr/BIP-340, BLS signatures, ML-DSA, SLH-DSA	RSA PKCS #1 v1.5 signatures, DSA, Falcon/FN-DSA while FIPS 206 is still in development
Public-key encryption and KEMs	RSA-OAEP, HPKE, X25519, X448, ML-KEM	RSAES-PKCS1-v1_5, ECIES variants, HQC while standardization is still in progress
Secret sharing	Shamir secret sharing, additive secret sharing, Feldman VSS, Pedersen VSS	Naive share splitting

Structured primitives

Primitive page	Include first	Include later or mention as caution
Homomorphic encryption	BFV, BGV, CKKS, TFHE/FHEW	Parameter-selection examples and bootstrapping costs
Homomorphic commitments	Pedersen vector commitments, KZG commitments, inner-product-argument commitments	Trusted-setup and pairing-specific caveats
Threshold cryptography	FROST, threshold BLS, threshold ECDSA families, distributed key generation (DKG)	Implementation-specific MPC signing protocols
Accumulators and authenticated data structures	Merkle trees, sparse Merkle trees, RSA accumulators, bilinear accumulators, Verkle/KZG vector commitments	Dynamic accumulator update costs
Timelocks and VDFs	Wesolowski VDF, Pietrzak VDF, timelock encryption from repeated squaring	Centralized delay services
Functional encryption	Identity-based encryption, attribute-based encryption, inner-product functional encryption	General functional encryption as mostly theoretical

Proof systems

Proof-system page	Include first	Include later or mention as caution
Interactive proof building blocks	Sigma protocols, Schnorr identification, Fiat-Shamir transform	Rewinding assumptions and transcript ambiguity
SNARK families	Groth16, PLONK, Marlin, Sonic-style universal setup systems	Scheme-specific arithmetization details
Transparent proof systems	STARKs, FRI, DEEP-FRI	Parameter and hash choices
Inner-product systems	Bulletproofs, Halo-style accumulation, Nova-style folding	Proof-size and verification trade-offs

Proof-system page	Include first	Include later or mention as caution
Range proofs	Bulletproof range proofs, Pedersen commitment plus range proof patterns	Bit-decomposition pitfalls
Membership proofs	Merkle inclusion proofs, sparse Merkle proofs, RSA accumulator witnesses, KZG opening proofs	Non-membership proofs and update witnesses
Lookup and set arguments	Plookup-style lookups, permutation arguments, grand-product arguments	Circuit-specific soundness caveats

Protocols

Protocol page	Include first	Include later or mention as caution
Key establishment and secure channels	TLS 1.3, HPKE, Noise patterns, Signal X3DH and Double Ratchet	Legacy TLS and static key exchange
Multi-party computation	Yao garbled circuits, GMW, BGW, SPDZ, MASCOT, oblivious transfer extension	Fairness and abort-model variants
Secure aggregation	Bonawitz-style secure aggregation, Prio/Prio+	Small-cohort leakage and dropout handling
Oblivious pseudorandom functions	RFC 9497 OPRF/VOPRF/POPRF suites	Prime-order-group suites are quantum-vulnerable; application context binding matters.
Private set intersection	Diffie-Hellman PSI, OPRF-based PSI, circuit PSI	Cardinality-only PSI and malicious-security upgrades
Anonymous credentials	CL signatures / Idemix, BBS+ signatures, selective-disclosure JWT/VC patterns, ZK credential systems	Revocation and rare-attribute leakage
Mixnets	Chaumian mixnets, Sphinx packet format, Loopix-style mixnets	Timing and active tagging attacks
Electronic voting	Helios-style encrypted tallying, mixnet tallying, homomorphic tallying, coercion-resistant protocols	Receipt-freeness claims without a coercion model

Design patterns and systems

Pattern or system page	Include first	Include later or mention as caution
Commit-reveal	Hash commit-reveal, Pedersen commit-reveal, VDF-assisted delayed reveal	Abort and last-mover advantage
Anonymous membership	Semaphore-style Merkle membership, accumulator-based membership, nullifier-based rate limiting	Small anonymity sets
Anti-double-use nullifiers	Hash nullifiers, serial-number e-cash patterns, context-bound nullifiers	Cross-context linkability
Private aggregation	Secret-shared aggregation, Prio-style validation, differential privacy composition	Differencing attacks
Private payments	Zerocoin, Zerocash, Zcash Sapling/Orchard-style note systems	Ledger metadata leakage
ZK rollups	Groth16-based rollups, PLONK-ish rollups, STARK-based rollups	Data availability and prover centralization

Editorial policy

Algorithm coverage should be practical rather than exhaustive. Prefer:

- one concept page for the primitive or protocol;
- a comparison table for common concrete schemes;
- a dedicated page only when the scheme has distinct assumptions, failure modes, deployment status, or composition risks.

Legacy or broken algorithms can be included when readers need to recognize them, but they should be labeled as legacy, deprecated, or unsafe for new systems.

Further reading

- NIST, FIPS 180-4: Secure Hash Standard.
- NIST, FIPS 202: SHA-3 Standard.
- NIST, FIPS 197: Advanced Encryption Standard.
- NIST, FIPS 186-5: Digital Signature Standard.
- NIST, Post-Quantum Cryptography standardization.
- RFC 5869, HMAC-based Extract-and-Expand Key Derivation Function.
- RFC 7748, Elliptic Curves for Security.
- RFC 8032, Edwards-Curve Digital Signature Algorithm.
- RFC 8439, ChaCha20 and Poly1305 for IETF Protocols.
- RFC 9180, Hybrid Public Key Encryption.
- RFC 9380, Hashing to Elliptic Curves.

Diagram Authoring

Diagrams in the atlas should be source-controlled as structured YAML and rendered into static assets.

Policy

- Edit diagram sources in `book/data/diagrams/*.yml`.
- Validate diagram YAML against `book/schemas/diagram.schema.json`.
- Generate SVG assets with `npm run generate:diagrams`.
- Use generated SVG files from `book/static/img/diagrams/` in pages.
- Keep generated Mermaid files in `book/static/diagrams/` when a simple flowchart view is useful for review.
- Do not hand-edit generated SVG or Mermaid outputs.

When to add a diagram

Prefer a diagram when the page explains:

- taxonomy or category boundaries;
- dependency chains between assumptions, primitives, proof systems, protocols, and systems;
- protocol flows with multiple participants or public artifacts;
- composition paths where a component contributes only one narrow guarantee;
- failure modes caused by metadata, setup, trust, or omitted checks.

Source format

Each YAML file defines:

- a stable `id`;
- a human-readable `title` and `description`;
- a grid layout;
- typed nodes with labels, details, and positions;
- directed edges;
- output paths for generated SVG and optional Mermaid.

Example:

```yaml
id: example-flow
title: "Example flow"
description: A short explanation for readers and screen readers.
layout:
  width: 960
  height: 480
  columns: 3
  rows: 2
  node_width: 244
  node_height: 72
outputs:
  svg: static/img/diagrams/example-flow.svg
  mermaid: static/diagrams/example-flow.mmd
nodes:
  - id: input
    label: Input
    detail: Public data
    kind: actor
    column: 1
    row: 1
  - id: proof
    label: Proof
    detail: Verifiable artifact
    kind: proof
    column: 2
    row: 1
edges:
  - from: input
    to: proof
```

Current diagram sources

- `book/data/diagrams/assumption-stack.yml`
- `book/data/diagrams/nullifier-flow.yml`
- `book/data/diagrams/private-voting-composition.yml`
- `book/data/diagrams/taxonomy-flow.yml`
- `book/data/diagrams/zero-knowledge-proof-flow.yml`

Post-Quantum Posture

Post-quantum posture describes how a primitive, protocol, or system is expected to behave against an adversary with a large cryptographically relevant quantum computer.

This label is not a deployment recommendation. It is an editorial signal that tells the reader which parts of a design are likely to need replacement or closer review during post-quantum migration.

Labels

Label	Meaning
vulnerable	The usual construction is broken or seriously weakened by known quantum algorithms.
plausible	The construction family is commonly treated as a post-quantum candidate, assuming appropriate parameters and implementation.
depends	The posture depends on the concrete instantiation, parameter set, or supporting primitives.
unknown	The posture is not established enough for this atlas to classify it.
not-applicable	The page describes a goal, pattern, or organizational model rather than a cryptographic construction.

Quick matrix

Concept family	Working posture	Notes
Hash functions	plausible	Depends on output length and security target.
Symmetric encryption and MACs	plausible	Depends on key sizes and concrete security margins.
RSA and finite-field discrete logarithm	vulnerable	Affects RSA encryption, RSA signatures, and related assumptions.
Elliptic-curve discrete logarithm	vulnerable	Affects ECDSA, EdDSA, Schnorr-style signatures, and many commitments.
Pedersen commitments	vulnerable	Usually discrete-logarithm based.
Pairing-based SNARKs	vulnerable	Pairings rely on elliptic-curve discrete-logarithm style assumptions.
STARK-style proof systems	plausible	Often hash-based and transparent, but the full system still depends on parameters and hash choices.
Bulletproof-style range proofs	vulnerable	Usually discrete-logarithm based.
Lattice-based encryption and FHE	plausible	Depends on scheme, parameters, and implementation.
Functional encryption	depends	The posture is scheme-specific and often research-stage.
Secret sharing core	plausible	Information-theoretic sharing can be post-quantum, while authentication and transport layers may not be.

Concept family	Working posture	Notes
Nullifiers	depends	Usually hash-based, but the surrounding credential or proof system may not be post-quantum.
VDFs and timelocks	depends	Many constructions rely on assumptions whose post-quantum posture is construction-specific.
Design patterns	not-applicable	The pattern inherits posture from the primitives and protocols used.

How to use this label

When a page says "depends," ask:

- Which concrete scheme is being used?
- Which mathematical assumption does it rely on?
- Is any setup, signature, encryption, or proof layer quantum-vulnerable?
- Are parameters sized for a post-quantum security target?
- Is the claim backed by a standard, peer-reviewed analysis, or only by early research?

Current standards context

As of June 4, 2026, NIST's principal post-quantum standards are FIPS 203 for ML-KEM, FIPS 204 for ML-DSA, and FIPS 205 for SLH-DSA. NIST also states that Falcon/FN-DSA and HQC were selected for ongoing standardization, but those are not the same as the three finalized FIPS standards.

This atlas should still classify broad concepts conservatively, because a concept such as "signature" or "encryption" can be instantiated with either quantum-vulnerable or post-quantum schemes.

Further reading

- NIST: Post-Quantum Cryptography FIPS Approved
- NIST CSRC: Post-Quantum Cryptography Project
- NIST, NIST Selects HQC as Fifth Algorithm for Post-Quantum Encryption
- NIST NCCoE: Frequently Asked Questions about Post-Quantum Cryptography
- Shor, "Algorithms for Quantum Computation; Discrete Logarithms and Factoring."
- Grover, "A Fast Quantum Mechanical Algorithm for Database Search."
- NIST FIPS 203, FIPS 204, and FIPS 205.

Confidence Models

A confidence model states where the reader's confidence is supposed to come from.

In cryptographic systems, confidence may come from a mathematical assumption, public verification, a quorum of independent parties, at least one honest participant, an honest majority, a trusted issuer, or operational controls. These are not interchangeable.

Common models

Model	Meaning	Typical failure
mathematical-assumption	Security rests mainly on a hardness assumption and implementation correctness.	The assumption is broken, parameters are weak, or implementation leaks secrets.
public-verifiability	Anyone can check the relevant proof, transcript, or commitment.	The public statement is incomplete or the verifier checks the wrong property.
trusted-setup	A setup phase must be generated honestly or securely destroyed.	Toxic waste, biased parameters, or unverifiable setup.
trusted-issuer	One issuer or authority controls eligibility, credentials, or identity binding.	The issuer misissues, logs, censors, or deanonymizes users.
one-honest-party	Privacy or correctness holds if at least one party in a set behaves honestly.	All parties collude, or the honest party is excluded.
t-of-n-threshold	A property fails only when at least `t` out of `n` parties collude or are unavailable.	Enough parties collude, lose keys, or are coerced.
honest-majority	Security requires more honest than corrupt participants.	The adversary controls the majority or network scheduling violates the model.
non-collusion	Multiple services are assumed not to share data or keys.	Services collude, merge, are subpoenaed together, or share infrastructure.
client-side-secret	A user secret or device key anchors the guarantee.	The secret is stolen, shared, backed up insecurely, or coerced from the user.
external-timing	Timing or delay assumptions are part of the guarantee.	Hardware advantage, network latency, or denial of service changes the effective timing.

Quick matrix

Concept or protocol	Typical confidence model	What to ask
Hash commitment	mathematical-assumption	Is the message high entropy or properly randomized?
Pedersen commitment	mathematical-assumption plus public parameters	Who generated the generators, and can anyone know their discrete-log relation?
Pairing-based SNARK	trusted-setup or universal setup, depending on construction	What happens if setup toxic waste exists?
STARK-style proof	public-verifiability and transparent setup	Are hash assumptions and public inputs appropriate?
Bulletproof-style range proof	mathematical-assumption and public-verifiability	Is the proof bound to the correct commitment and range?

Concept or protocol	Typical confidence model	What to ask
Threshold decryption	t-of-n-threshold	How many trustees can collude before privacy fails?
Mixnet	one-honest-party plus batching	Is at least one mix honest, and is the anonymity set large enough?
MPC	honest-majority, dishonest-majority, or threshold model	Which corruption model does the protocol actually support?
Secure aggregation	threshold, dropout, and collusion model	How many clients or helper servers may collude?
Anonymous credentials	trusted-issuer plus holder secret	Can the issuer or verifier link presentations?
Nullifiers	client-side-secret plus public registry	Does the nullifier prove eligibility, or only non-reuse?
E-voting	threshold trustees, public bulletin board, and audit process	Which authorities can collude, censor, or create receipts?

How to write this in pages

Prefer concrete statements:

> Privacy holds if fewer than `t` of `n` trustees collude, the proof system statement is correct, and metadata outside the tally is handled separately.

Avoid vague statements:

> The system is decentralized and therefore trustless.

Further reading

- Canetti, "Universally Composable Security; A New Paradigm for Cryptographic Protocols."
- Shamir, "How to Share a Secret."
- Benaloh, "Verifiable Secret-Ballot Elections."

Metadata Leakage

Metadata leakage is information revealed outside the protected cryptographic payload.

Common leak classes

Leak	Examples	Typical mitigation
Identity	public keys, accounts, issuer records	anonymous credentials, mixnets, fresh identifiers
Timing	submission time, reveal time, response delay	batching, delays, cover traffic
Size	ciphertext length, proof size, message count	padding, fixed-size messages
Participation	who joined, dropped out, or abstained	cohort thresholds, aggregation policy
Context	election id, app id, public inputs	domain separation, minimal public inputs
Output	final aggregate, function result, tally	cohort-size rules, differential privacy, query limits

How to use this appendix

For every primitive, protocol, or system page, separate the cryptographic statement from the metadata statement. A zero-knowledge proof may hide a witness while the public inputs or transaction timing still identify the user.

Further reading

- Chaum, "Untraceable Electronic Mail, Return Addresses, and Digital Pseudonyms."
- Canetti, "Universally Composable Security; A New Paradigm for Cryptographic Protocols."

Threat-Model Checklist

Use this checklist before selecting primitives.

Questions

- What is the exact security goal: confidentiality, integrity, anonymity, unlinkability, verifiability, or coercion resistance?
- Which actors can be malicious, curious, offline, coerced, or compromised?
- Which parties must remain honest, non-colluding, available, or publicly verifiable?
- What metadata remains public even if cryptographic payloads are hidden?
- What setup, issuer, threshold, timing, or network assumptions are required?
- What happens on abort, dropout, lost keys, stale state, or disputed outputs?
- Which claims are mathematical, and which are operational or governance claims?
- Which components are quantum-vulnerable, and which merely inherit posture from another layer?

Output

A useful threat model should produce three lists: guarantees, non-goals, and leaks. If a claim cannot be placed in one of those lists, it is probably too vague to guide design.

Further reading

- Katz and Lindell, "Introduction to Modern Cryptography."
- Canetti, "Universally Composable Security; A New Paradigm for Cryptographic Protocols."